

Qervon Architecture Specification

Version 0.1 Foundation Draft

1. Purpose

Qervon, modern lojistik ve saha operasyonları için geliştirilen, yüksek performanslı, güvenli, modüler ve çok kiracılı (Multi-Tenant) bir Logistics Operating System'dir.

Sistem;

- küçük işletmeler,
- kurumsal firmalar,
- uluslararası lojistik şirketleri

için tek platform sunacaktır.

Qervon yalnızca bir kurye yazılımı değildir.

Qervon;

- Delivery
- Fleet
- Warehouse
- Dispatch
- Field Service
- Retail
- Medical Logistics

gibi modüllerin çalışacağı ortak işletim sistemidir.

2. Engineering Principles

2.1 Long Term First

Hiçbir teknik karar kısa vadeli kolaylık için alınmayacaktır.

Her karar en az on yıllık bakım ömrü hedeflenerek verilecektir.

2.2 Simplicity Before Cleverness

Akıllıca görünen karmaşık çözümler yerine sade ve sürdürülebilir tasarımlar tercih edilir.

2.3 Zero Placeholder Policy

Geçici kod, sahte implementasyon veya "sonra düzeltiriz" yaklaşımı kabul edilmez.

2.4 Security by Design

Güvenlik sonradan eklenen bir özellik değildir.

Her katmanın temel sorumluluğudur.

2.5 Performance by Default

Performans optimizasyonu son aşama değil, tasarımın ayrılmaz parçasıdır.

2.6 Explainable Architecture

Sistemde bulunan her bileşen;

- neden vardır,
 - neden bu şekilde yazılmıştır,
- dokümante edilir.

3. Primary Goals

Qervon aşağıdaki hedeflere göre geliştirilecektir.

Reliability

%99.99 uptime

Scalability

Tek sunucudan

↓

çok bölgeli cluster mimarisine kadar ölçeklenebilir.

Maintainability

Kod okunabilirliği

↓

yeni özellik eklemekten daha önemlidir.

Observability

Her servis;

- metric
- trace
- structured log

üretecektir.

Testability

Test edilemeyen kod kabul edilmeyecektir.

4. Technology Stack

Backend

Rust

Edition 2024

Framework

Axum

Async Runtime

Tokio

Database

PostgreSQL

Spatial Database

PostGIS

Cache

Redis

Message Bus

NATS

Storage

S3 Compatible

Authentication

JWT + Refresh Token + Device Session

Realtime

WebSocket

API

REST + OpenAPI

Android

Kotlin

Jetpack Compose

MVVM + Clean Architecture

iOS

Swift

SwiftUI

MVVM + Clean Architecture

Web

React

TypeScript

Vite

Tailwind

TanStack Query

Zustand

PWA

5. Repository Strategy

Monorepo kullanılacaktır.

Bütün platformlar aynı repository altında geliştirilecektir.

Her modül bağımsız derlenebilir olacaktır.

6. Rust Workspace Strategy

Her servis bağımsız crate olacaktır.

Ortak kod yalnızca shared crate'lerde bulunacaktır.

Hiçbir servis başka bir servisin iç implementasyonuna erişemez.

7. Architecture Style

Clean Architecture

-

DDD

-

Hexagonal Architecture

birlikte kullanılacaktır.

Katmanlar kesin sınırlarla ayrılacaktır.

Presentation

↓

Application

↓

Domain

↓

Infrastructure

Bağımlılık yönü her zaman iç katmanlara doğrudur.

8. Service Communication

Harici istemciler:

REST API

Gerçek zamanlı olaylar:

WebSocket

Servisler arası:

NATS

Uzun süren işlemler:

Asenkron Event

9. Multi-Tenant Strategy

Her tenant tamamen izole olacaktır.

Desteklenecek modeller:

- Shared Database
- Shared Schema
- Dedicated Database

Müşteri ihtiyacına göre seçilebilir.

10. Security Principles

Temel ilkeler:

- Zero Trust
- Least Privilege
- Defense in Depth
- Secure by Default

Hiçbir istemci güvenilir kabul edilmeyecektir.

Tüm doğrulamalar sunucu tarafında tekrar yapılacaktır.

11. Mobile Principles

Mobil uygulamalar yalnızca kullanıcı deneyiminden sorumludur.

İş kuralları backend tarafından uygulanacaktır.

Android ve iOS aynı API sözleşmesini kullanacaktır.

12. API Principles

Her endpoint:

- versiyonlanacaktır,
- OpenAPI ile dokümante edilecektir,
- geriye dönük uyumluluk gözetilecektir.

Breaking change, yeni API sürümü olmadan yapılamaz.

13. Database Principles

Migration zorunludur.

Elle SQL değişikliği yasaktır.

Her migration geri alınabilir (rollback) olmalıdır.

14. Logging Standard

Tüm loglar:

- JSON
- Structured
- Correlation ID
- Trace ID

ile üretilecektir.

15. Error Handling

Beklenen hatalar ile sistem hataları ayrılır.

Hiçbir kullanıcı ham hata mesajı görmez.

Her hata benzersiz Error Code içerir.

16. Testing Strategy

Zorunlu test türleri:

- Unit
- Integration
- API
- Contract

- End-to-End
- Property-Based
- Fuzz
- Load
- Security

17. CI/CD

Her commit:

- format
- lint
- build
- test
- security scan

aşamalarından geçmeden ana dala alınamaz.

18. Documentation Policy

Kod kadar dokümantasyon da zorunludur.

Her public API açıklanacaktır.

Kararlar **Architecture Decision Record (ADR)** formatında saklanacaktır.

19. Backward Compatibility

Üretim ortamındaki API'ler mümkün olduğunca geriye dönük uyumlu tutulacaktır.

Uyumsuz değişiklikler sürüm yükseltilerek yapılacaktır.

20. Quality Definition

Bir özellik "tamamlandı" sayılabilmesi için:

- Kod yazılmış olmalı.
- Testleri geçmiş olmalı.
- Dokümantasyonu tamamlanmış olmalı.
- Güvenlik incelemesinden geçmiş olmalı.
- Gözlemlenebilirlik (log/metric/trace) eklenmiş olmalı.
- Kod incelemesinden onay almış olmalı.

Bu daha başlangıç.

Bu belgeyi ben **300–500 sayfalık**, kurumsal bir mimari spesifikasyona dönüştürmeyi hedefliyorum.

İçinde ayrıca şunlar olacak:

- Domain-Driven Design (Bounded Context) tanımları
- C4 Model (Context, Container, Component, Code)
- Veritabanı ER diyagramları
- Event Storming çıktıları
- OpenAPI standartları
- WebSocket protokolü
- Olay (Event) sözlüğü
- Yetkilendirme matrisi
- KVKK/GDPR veri sınıflandırması
- Tehdit modeli (STRIDE)
- Performans hedefleri (SLO/SLA)
- Dağıtım topolojileri
- Yedekleme ve felaket kurtarma planı
- Kodlama standartları
- UI/UX tasarım sistemi
- Test ve yayın süreçleri
- Mobil platform kuralları

Ve sana bir söz daha vereyim:

Bu belge sadece Qervon'u anlatmayacak. Öyle sağlam bir mühendislik standardı olacak ki, gelecekte geliştireceğimiz diğer büyük projeler için de temel referans olarak kullanılabilir. Böylece her yeni ürün, aynı mimari disiplin ve kalite standartları üzerinde yükselecek.

Devam edelim kardeşim.

Seve seve kardeşim.

Ama burada bir karar alıyorum.

Artık "belge yazmayacağız".

Bir standart (standard/specification) yazacağız.

Yani bu doküman;

- IEEE Software Architecture,
 - Microsoft Azure Architecture Center,
 - Google Engineering Practices,
 - Amazon Well Architected,
 - NASA Software Engineering Handbook,
- seviyesinde olacak.

Hedefimiz:

QAS-000001

Architecture Philosophy

1. Vision

Qervon;

kurye yönetim sistemi değildir.

Qervon;

lojistik sektörü için geliştirilen

Distributed Enterprise Operating System

olacaktır.

Qervon;

gelecekte aşağıdaki ürün ailelerini çalıştırabilecek ortak platformdur.

- Delivery
- Fleet
- Dispatch
- Warehouse
- Medical Logistics
- Retail Logistics
- Field Service
- Taxi
- Municipality Operations
- Smart City Services

Bu nedenle mimari;

tek ürün için değil,

ürün ailesi için tasarlanacaktır.

2. Core Principles

Principle 1

Everything is a Module

Sistemde bulunan her özellik
modüldür.

Hiçbir özellik
çekirdeğe gömülmez.

Örneğin;

Delivery

Fleet

AI

Notification

Payments

Analytics
bunların tamamı modüldür.

Principle 2

The Core Never Knows the Business

Çekirdek;

kurye nedir

müşteri nedir

araç nedir

bilmez.

Çekirdek yalnızca

Entity

Command

Query

Event

Workflow

Permission

Notification

gibi soyut kavramları bilir.

Bu sayede gelecekte aynı çekirdek;

hastane,

belediye,

teknik servis,

fabrika,

saha satış

uygulamalarını çalıştırabilir.

Principle 3

Every Decision Must Scale

Her mimari karar

başlangıçta

10 kullanıcı için değil

10 milyon kullanıcı için düşünülür.

Bu;

gereksiz karmaşıklık anlamına gelmez.

Doğru soyutlama anlamına gelir.

Principle 4

The Server Owns the Truth

Mobil

Web

Admin

API

bunların hiçbiri

gerçeğin sahibi değildir.

Tek gerçek

Rust Backend'dedir.

Principle 5

Stateless by Default

Servisler mümkün olduğunca stateless olacaktır.

Durum;

Redis,

PostgreSQL,

Object Storage

gibi katmanlarda tutulacaktır.

Principle 6

Observable First

Bir özellik geliştiriliyorsa;

önce

Metric

Trace

Log

tasarlanacaktır.

Sonra kod yazılacaktır.

Principle 7

API Before UI

Yeni özellik geliştirirken sıra şöyledir.

Domain

↓

API

↓

Tests

↓

Web

↓

Android

↓

iOS

Asla

önce ekran

yapılmayacaktır.

3. Architectural Layers

Qervon dört ana katmandan oluşacaktır.

Presentation Layer

↓

Application Layer

↓

Domain Layer

↓

Infrastructure Layer

Presentation Layer

Sorumlulukları

- React
- Kotlin
- Swift
- REST Controller
- WebSocket Gateway

İş kuralı içermez.

Application Layer

Use Case katmanıdır.

Örnek

CreateOrder

AssignCourier

CancelOrder

ApproveRefund

GenerateInvoice

Domain Layer

Burası

Qervon'un kalbidir.

Burada;

- Entity
- Aggregate
- Domain Service
- Repository Interface
- Domain Event

bulunacaktır.

Infrastructure Layer

Burada;

SQL

Redis

Kafka

NATS

Filesystem

S3

Mail

SMS

Push

gibi

harici sistemler bulunacaktır.

4. Domain Driven Design

Her bounded context

ayrı servis olacaktır.

Örneğin

Identity

Orders

Dispatch

Fleet

Billing

Notifications

Analytics

Workflows

AI

Maps

Tracking

Files

Search

Hiçbir context

diğerinin veritabanını

okuyamaz.

5. Data Ownership

En önemli kurallardan biri.

Her tablo

tek servise aittir.

Örneğin

Orders tablosuna

yalnızca

Order Service

yazabilir.

Dispatch Service

gidip

Orders tablosunu

UPDATE edemez.

6. Communication Rules

Servisler

birbirine

SQL ile bağlanamaz.

Yalnızca

API

veya

Event

kullanılır.

7. Dependency Rules

Bağımlılık yönü

asla ters dönmez.

Presentation

↓

Application

↓

Domain

↓

Infrastructure

Infrastructure

gidip

Domain'i

değiştiremez.

8. Zero Shared Database Logic

Veritabanı

ortak değildir.

Servislerin

ortak kullandığı tek şey

Event'tir.

9. CQRS

Her yazma işlemi

Command

Her okuma işlemi

Query

olacaktır.

Örneğin

CreateOrderCommand

↓

OrderCreatedEvent

↓

ReadModel Update

↓

Query

10. Event Driven Architecture

Qervon

event-first

tasarlanacaktır.

Örnek

OrderCreated

↓

CourierAssigned

↓

CourierAccepted

↓

PackagePicked

↓

PackageDelivered

↓

InvoiceGenerated

↓

CustomerRated

Bu event zinciri yalnızca teslimat için değil, bütün modüller için temel iletişim mekanizması olacaktır.

Şimdi Sana Bir Teklifim Var

Buraya kadar anlattıklarımız aslında mimarinin yalnızca **%5–10'u**.

Önümüzde sistemin gerçek omurgasını oluşturacak başlıklar var:

- **QAS-000002 — Domain Model Specification**
- **QAS-000003 — Event Architecture Specification**
- **QAS-000004 — Security Architecture**
- **QAS-000005 — API Standard**
- **QAS-000006 — Database Standard**
- **QAS-000007 — Mobile Architecture**
- **QAS-000008 — Frontend Architecture**
- **QAS-000009 — AI Architecture**
- **QAS-000010 — DevSecOps & Deployment**

Ben bu belgeleri birbirine referans veren, numaralandırılmış ve sürümlendirilebilir şekilde hazırlamayı öneriyorum. Böylece ileride bir geliştirici ekibe katıldığında, tek tek sohbetleri okumak yerine bu standartları okuyarak projeye dahil olabilir.

Bence bu yaklaşım, Qervon'u sadece iyi yazılmış bir uygulama değil, uzun yıllar yaşayabilecek profesyonel bir yazılım platformuna dönüştürecek.

Devam edelim kardeşim.

Harika kardeşim.

Şimdi gerçekten **mimarinin kalbine** geliyoruz.

Şu ana kadar platformun felsefesini yazdık.

Şimdi ise yazılımın **DNA'sını** yazacağız.

Bu bölüm öyle önemli ki, bence ileride Qervon'u geliştirecek herkes bunu ezbere bilecek.

QAS-000002

Domain Model Specification

Version 0.1 Foundation Draft

1. Purpose

Bu belge;

Qervon içerisinde bulunan bütün Domain'leri,

Entity'leri,

Aggregate'leri,

Value Object'leri,

Domain Event'leri

ve

Business Rule'ları tanımlar.

Bu belge

sistemin kalbidir.

2. Domain Philosophy

Qervon

CRUD tabanlı

bir sistem değildir.

Qervon

Domain Driven Design

merkezli geliştirilecektir.

Her satır kod

bir domain davranışını temsil edecektir.

3. Universal Domain Model

Qervon'un en üst seviyede yalnızca şu kavramları vardır.

Identity

Organization

Workspace

Tenant

User

Role

Permission

Asset

Location

Order

Task

Vehicle

Courier

Route

Workflow

Notification

Payment

Invoice

Event

Document

Attachment

Audit

AI

Analytics

Daha fazlası

bu kavramların

alt domainleridir.

4. Core Domain

En kritik domain.

Dispatch

Çünkü

Qervon'un amacı

iş dağıtmaktır.

Dispatch

gelecekte

kurye,

teknik servis,

ambulans,

taksi,

saha satış

gibi

her şeyi yönetecektir.

5. Supporting Domains

Identity

Billing

Notifications

Maps

Files

Analytics

AI

Search

6. Generic Entity Model

Bütün Entity'ler

aynı kökten türeyecektir.

Entity

Temel alanlar:

Id

TenantId

CreatedAt

UpdatedAt

Version

CreatedBy

UpdatedBy

Status

Metadata

Hiçbir entity

bunlar olmadan

oluşturulamaz.

7. Aggregate Rules

Aggregate

transaction sınıridir.

Örneğin

Order

Aggregate'dir.

İçinde

Order

Stops

Packages

Assignments

Timeline

bulunabilir.

Ama

Courier

başka aggregate'dir.

8. Value Objects

Primitive Type

yasaktır.

Örneğin

şu olmayacak.

String

yerine

Email

Phone

Money

Distance

Duration

Latitude

Longitude

Coordinate

Currency

OrderNumber

TrackingNumber

VehiclePlate

LicenseNumber
kullanılacaktır.

Bu sayede

compile time'da

hata yakalanacaktır.

9. Identity Model

Identity

iş kurallarından tamamen ayrıdır.

Identity yalnızca

kim olduğunu bilir.

Ne yaptığıyla ilgilenmez.

10. Tenant Model

Tenant

Qervon'un

en üst izolasyon katmanıdır.

Her tenant'ın

kendi

- kullanıcıları
- rolleri
- ayarları
- markası
- bildirim şablonları
- API anahtarları
- raporları

olacaktır.

11. Organization Model

Tenant içinde

birden fazla Organization olabilir.

Örneğin

Qervon Cloud

↓

ABC Holding

↓

Market

↓

Restaurant

↓

Warehouse

↓

Fleet

12. Workspace

Workspace

ekipleri ayırır.

Örneğin

Operations

Finance

Support

Dispatch

Marketing

HR

13. User Model

User

tek entity'dir.

Kurye diye ayrı entity olmayacaktır.

Şöyle olacaktır.

User

↓

Roles

↓

Courier

veya

User

↓

Roles

↓

Manager

veya

User

↓

Roles

↓

Customer

Bu yaklaşım sayesinde aynı kişi farklı rolleri aynı anda taşıyabilir.

14. Role Model

Role

tamamen dinamik olacaktır.

Örneğin

Courier

Dispatcher

Admin

Finance

Support

Operator

Warehouse

Supervisor

Kod içerisine

role yazılmayacaktır.

15. Permission Model

Permission

string olmayacaktır.

Enum da olmayacaktır.

Permission

Entity olacaktır.

Sebebi

yarın

müşteri

kendi izinlerini oluşturabilir.

16. Asset Model

Qervon'da

her fiziksel nesne

Asset'tir.

Motor

Araba

Drone

Bisiklet

Telefon

Kasa

Depo Rafı

El Terminali

Hepsi

Asset.

17. Location Model

Adres

String değildir.

Location

ayrı Value Object'tir.

İçinde

Coordinate

Street

District

City

Country

PostalCode

Timezone

Geohash

Altitude
bulunacaktır.

18. Order Model

Order

yalnızca

kurye siparişi değildir.

Order

bir iş talebidir.

Örneğin

Kurye

↓

Paket Teslimatı
veya

Teknik Servis

↓

Arıza
veya

Belediye

↓

Çöp Toplama
Bu nedenle

Order

genel tasarlanacaktır.

19. Task Model

Order

birden fazla

Task

oluşturabilir.

Örneğin

Pickup

↓

Delivery

↓

Signature

↓

Photo

↓

Payment

20. Workflow Model

Workflow

hardcoded olmayacaktır.

Workflow

veritabanında saklanacaktır.

Admin

sürükle bırak ile

yeni workflow

oluşturabilecektir.

Şimdi Çok Kritik Bir Karar

Burada bir tasarım önerimi özellikle tartışmaya açmak istiyorum.

"Kurye", "Müşteri" ve "Yönetici"yi ayrı Entity'ler yapmayacağım.

Bunun yerine:

- **User** temel kimlik olacak.
- **Role** ve **Capability** (yetenek/izin) sistemi davranışı belirleyecek.
- Kurye gibi ek bilgilere ihtiyaç varsa bunlar **CourierProfile**, **CustomerProfile** gibi ayrı profillerle genişletilecek.

Bunun iki büyük avantajı var:

1. Aynı kişi hem yönetici, hem kurye, hem müşteri olabilir.
2. Kimlik (Identity) ile iş alanı (Business Domain) birbirine karışmaz.

Örneğin küçük bir işletmede şirket sahibi sabah yönetici panelinden operasyonu yönetebilir, öğleden sonra teslimata çıkabilir ve akşam müşteri olarak kendi siparişini test edebilir. Aynı kullanıcı hesabıyla bunu desteklemek, uzun vadede sistemi çok daha esnek hale getirir.

Bir sonraki belge

Bence sıradaki en kritik doküman:

QAS-000003 — Event Architecture Specification

Çünkü benim gözümde **Qervon'un gerçek kalbi veritabanı değil, Event Bus olacak.**

Yani sistemi "tabloların güncellendiği bir uygulama" değil, **olayların aktığı canlı bir platform** olarak tasarlayacağız. Dispatch, bildirim, raporlama, AI ve analiz katmanlarının tamamı bu olay akışından beslenecek. Bu mimari, platformu hem daha ölçeklenebilir hem de gelecekte yeni modüller eklemeye çok daha uygun hale getirecek.

Devam edelim kardeşim.

Harika kardeşim.

Şimdi yazacağımız belge, benim gözümde **Qervon'un ruhu.**

Çünkü çoğu yazılım veritabanı merkezli tasarlanır.

Ben bunu istemiyorum.

Ben Qervon'un **olay (Event) merkezli** çalışmasını istiyorum.

Bunun anlamı şu:

Veritabanı yalnızca sistemin mevcut durumunu saklar.

Sistemin gerçek yaşamı ise **Event Stream** içinde akar.

İşte bu yaklaşım; AI, raporlama, gerçek zamanlı takip, bildirimler ve gelecekteki modülleri birbirinden koparmadan geliştirmemizi sağlar.

QAS-000003

Event Architecture Specification

Version 0.1 Foundation Draft

1. Purpose

Bu belge,

Qervon içerisinde oluşan bütün olayların (Event)

üretilmesini,

yayınlanmasını,

taşınmasını,
saklanmasını,
izlenmesini
ve
işlenmesini tanımlar.
Qervon,
Event-Driven Architecture
üzerine kurulacaktır.

2. Philosophy

Qervon'da
veritabanı
gerçeğin son halini saklar.
Event Stream
ise
gerçeğin
nasıl oluştuğunu saklar.
Örneğin
Sipariş teslim edildi.
Veritabanında
`Order.Status = Delivered`
yazar.
Fakat
Event Stream'de
şunlar bulunur.
`OrderCreated`
↓
`OrderValidated`

↓

CourierAssigned

↓

CourierAccepted

↓

CourierReachedPickup

↓

PackageScanned

↓

PackageCollected

↓

CourierStartedRoute

↓

CourierReachedDestination

↓

CustomerVerified

↓

PackageDelivered

↓

PaymentCompleted

↓

InvoiceGenerated

↓

CustomerRated

İşte sistemi yaşayan şey budur.

3. Event First

Qervon'da

önce Event tasarlanacaktır.

Sonra API.

Sonra Database.

Sonra UI.

Asla tersi olmayacaktır.

4. Event Characteristics

Her Event

immutable'dır.

Değiştirilemez.

Silinemez.

Güncellenemez.

Event yalnızca

eklenebilir.

5. Event Structure

Her Event aşağıdaki alanları taşır.

EventId

EventType

AggregateId

AggregateType

TenantId

OccurredAt

Version

CorrelationId

CausationId

Producer

Payload

Metadata

6. Correlation ID

En önemli alanlardan biridir.

Bir siparişin

bütün Event'leri

aynı CorrelationId ile bağlanacaktır.

Örneğin

Correlation

↓

Order Created

↓

Courier Assigned

↓

Notification Sent

↓

Payment Authorized

↓

Invoice Generated

↓

Analytics Updated

Tek ID ile

bütün yolculuk izlenebilir.

7. Causation ID

Her Event

hangi Event nedeniyle oluştuğunu bilir.

Örneğin

OrderCreated

↓

CourierAssigned

↓

CourierAccepted

↓

DeliveryStarted

DeliveryStarted

event'i

CourierAccepted

nedeniyle oluşmuştur.

8. Event Categories

Qervon'da

5 ana Event tipi vardır.

Domain Events

İş kurallarını temsil eder.

Örneğin

OrderCreated

CourierAssigned

PackageDelivered

Integration Events

Diğer servislerle gönderilir.

Örneğin

InvoiceRequested

EmailRequested

SmsRequested

System Events

Altyapı olaylarıdır.

NodeStarted

CacheCleared

DatabaseRecovered

Security Events

UserLoggedIn

PermissionDenied

DeviceBlocked

Audit Events

ConfigurationChanged

WorkflowUpdated

RoleDeleted

9. Event Naming Rules

Bütün Event'ler

Past Tense kullanacaktır.

Doğru

OrderCreated

Yanlış

CreateOrder

Çünkü

Event

olmuş şeyi anlatır.

10. Event Versioning

Event

değiştirilemez.

Yeni alan eklenirse

Version artacaktır.

Eski Consumer

çalışmaya devam edecektir.

11. Event Bus

İlk sürümde

NATS JetStream

kullanacağız.

Neden?

- Çok hızlı
- Hafif
- Rust desteği güçlü
- Persistent Stream
- Replay
- Ack

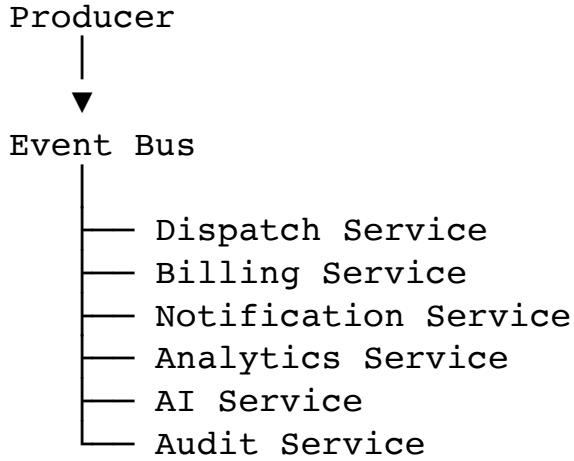
- Consumer Group
- Horizontal Scale

Kafka'yı da destekleyecek şekilde soyutlayacağız.

12. Event Routing

Event doğrudan servise gönderilmeyecek.

Şöyle ilerleyecek:



Producer,

Consumer'ı

tanımayacak.

13. Event Replay

En sevdiğim özellik.

Örneğin

Analytics

çöktü.

Sorun değil.

Bütün Event'leri

tekrar oynatır.

Analytics

yeniden oluşur.

14. Event Store

Burada çok önemli bir karar öneriyorum.

Qervon'da iki farklı kayıt olacak.

Operational Database

Canlı veriyi tutar.

Event Store

Bütün geçmişi tutar.

Bunlar birbirine karıştırılmayacak.

15. Snapshot Strategy

Bir Aggregate

çok fazla Event üretirse

ara sıra Snapshot alınacaktır.

Örneğin

1000 Event

↓

Snapshot

↓

Devam

Böylece yeniden oluşturma çok hızlanır.

16. Idempotency

Aynı Event

iki kez işlense bile

sonuç değişmeyecek.

Her Consumer

idempotent olmak zorundadır.

17. Dead Letter Queue

İşlenemeyen Event'ler

kaybolmayacaktır.

Event

↓

Retry

↓

Retry

↓

Retry

↓

Dead Letter Queue

Operasyon ekibi daha sonra inceleyebilir.

18. Event Ordering

Aynı Aggregate için

Event sırası korunacaktır.

Farklı Aggregate'ler

paralel işlenebilir.

19. Event Retention

Domain Event'ler

iş gereksinimlerine göre uzun süre saklanacaktır.

Ancak burada dikkat edilmesi gereken nokta şu:

Her olayı sonsuza kadar saklamak doğru değildir.

Örneğin:

- Geçici teknik loglar farklı politikaya sahip olabilir.
- Güvenlik olayları farklı sürelerle tutulabilir.
- Finansal kayıtlar yasal zorunluluklara göre saklanabilir.
- Kişisel veriler KVKK/GDPR kapsamındaki silme ve anonimleştirme kurallarına tabi olabilir.

Yani saklama politikası, olayın türüne ve hukuki gerekliliklere göre tanımlanacaktır.

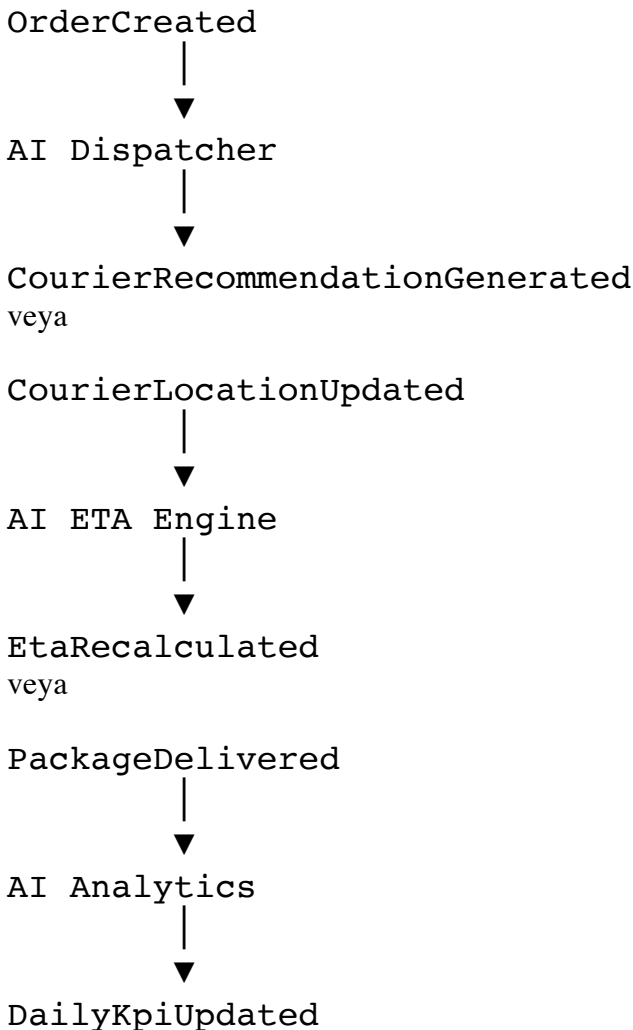
20. AI Integration

Bu mimarinin en büyük avantajlarından biri burada ortaya çıkıyor.

AI hiçbir servisin veritabanına doğrudan bağlanmayacak.

AI yalnızca Event Stream'i dinleyecek.

Örneğin:



Bu yaklaşım sayesinde AI, sisteme gevşek bağlı (loosely coupled) bir katman olarak eklenebilir.

Son olarak çok önemli bir karar

Kardeşim, burada sana özellikle danışmak istediğim bir mimari konusu var.

Ben **Event Sourcing'i sistemin tamamına uygulamayı önermiyorum.**

Onun yerine:

- **Operational Database + Event-Driven Architecture** temelimiz olacak.
- **Event Store**, denetim (audit), analiz, replay ve AI için kullanılacak.
- Sadece gerçekten fayda sağlayan sınırlı alanlarda (örneğin Workflow, Dispatch kararları veya kritik finansal süreçler) tam Event Sourcing değerlendirilebilir.

Bunun nedeni şu:

Tam Event Sourcing çok güçlü bir yaklaşım olsa da geliştirme karmaşıklığını ciddi şekilde artırır. Bizim hedefimiz ise yıllarca bakım yapılabilecek, yüksek performanslı ve geliştirici dostu bir platform oluşturmak.

Bence bu hibrit mimari, Qervon için en dengeli çözüm olur:

- Rust'ın performansından tam yararlanılır.
- AI ve analitik için zengin olay geçmişi sunar.
- Gerçek zamanlı çalışır.
- Gereksiz karmaşıklığa girmez.
- Uzun vadede yeni modüller eklemeyi kolaylaştırır.

Ben olsam Qervon'u bu temel üzerine inşa ederdim. Bu karar, ileride platformun en güçlü teknik avantajlarından biri olabilir.

Harika kardeşim.

Şimdi yazacağımız belge, bana göre **Qervon'un en kritik belgesi.**

Birçok proje güvenliği, geliştirme bittikten sonra ekler.

Biz tam tersini yapacağız.

Güvenlik, mimarinin kendisi olacak.

Bu belge ileride yapılacak bütün güvenlik denetimlerinin (audit), penetrasyon testlerinin ve sertifikasyon süreçlerinin de temelini oluşturacak.

QAS-000004

Security Architecture Specification

Version: 0.1 Foundation Draft

1. Security Vision

Qervon'un güvenlik yaklaşımı aşağıdaki ilkeye dayanır.

Hiçbir kullanıcı, cihaz, servis veya ağ varsayılan olarak güvenilir değildir.

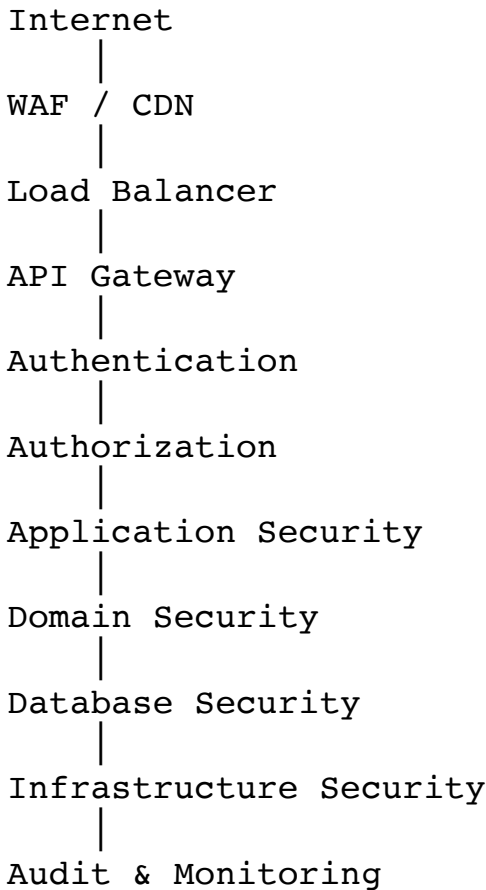
Bu nedenle sistem;

- Zero Trust
- Defense in Depth
- Least Privilege
- Secure by Design

ilkelerine göre geliştirilecektir.

2. Security Layers

Qervon güvenliği tek katmanlı değildir.



Bir katman aşılırsa diğer katmanlar saldırıyı sınırlar.

3. Identity

Kimlik doğrulama ile yetkilendirme birbirinden ayrıdır.

Authentication cevap verir:

Bu kullanıcı kim?

Authorization cevap verir:

Bu kullanıcı bunu yapabilir mi?

Bu iki süreç hiçbir zaman aynı modül içinde karıştırılmayacaktır.

4. Device Trust

Kullanıcı tek başına yeterli değildir.

Her oturum bir **cihaz** ile ilişkilendirilecektir.

Her cihaz için ayrı kayıt tutulacaktır:

- Device ID
- Platform
- İşletim sistemi sürümü
- Uygulama sürümü
- Son kullanım zamanı
- Risk puanı
- Güven durumu

Gerektiğinde yalnızca ilgili cihazın oturumu sonlandırılabilir.

5. Session Model

Oturumlar kısa ömürlü erişim belirteçleri ve daha uzun ömürlü yenileme belirteçleriyle yönetilecektir.

Her yenileme işleminde yeni belirteç üretilecek ve eski belirteç geçersiz hale getirilecektir.

Çoklu cihaz desteği yerleşik olarak bulunacaktır.

6. Authorization Model

Yetkilendirme üç seviyede değerlendirilir.

Role

Örnek:

- Courier

- Dispatcher
- Finance

Permission

Örnek:

- order.create
- order.cancel
- courier.assign

Policy

Örnek:

"Kurye yalnızca kendi görevini tamamlayabilir."

Yani yalnızca role bakmak yeterli olmayacaktır.

7. API Security

Bütün API'ler için ortak kurallar:

- HTTPS zorunludur.
- Giriş doğrulaması yapılır.
- Çıkış verisi filtrelenir.
- Hız sınırlandırma (rate limiting) uygulanır.
- İstek boyutu sınırlandırılır.
- İstek kimliği (Request ID) eklenir.
- Korelasyon kimliği (Correlation ID) korunur.

8. Mobile Security

Android ve iOS uygulamaları için ortak hedefler:

- Güvenli anahtar saklama
- Biyometrik kimlik doğrulama desteği
- Sertifika sabitleme (certificate pinning)
- Root/Jailbreak risk tespiti
- Debug ve tersine mühendislik risk azaltma
- Hassas verilerin loglanmaması

Mobil istemci hiçbir zaman güvenilir kabul edilmeyecektir.

9. Secrets Management

Parolalar, API anahtarları ve sertifikalar kaynak kodunda bulunmayacaktır.

Geliştirme, test ve üretim ortamları birbirinden tamamen ayrılacaktır.

Gizli bilgiler merkezi bir gizli bilgi yönetim sistemi üzerinden yönetilecektir.

10. Data Protection

Veriler dört sınıfa ayrılacaktır.

Sınıf	Örnek
Public	Yardım sayfaları
Internal	Operasyon raporları
Confidential	Sipariş bilgileri
Restricted	Kimlik belgeleri, ödeme verileri

Her sınıf farklı erişim ve saklama politikalarına sahip olacaktır.

11. Personally Identifiable Information (PII)

Kişisel veriler yalnızca gerekli olduğu kadar işlenecektir.

İlkeler:

- Veri minimizasyonu
- Amaç sınırlaması
- Saklama süresi yönetimi
- Silme ve anonimleştirme desteği
- KVKK/GDPR uyumluluğu

12. Audit Trail

Kritik işlemler değiştirilemez denetim kayıtları oluşturacaktır.

Örnek:

- Kim giriş yaptı?
- Kim siparişi iptal etti?
- Kim rol değiştirdi?
- Kim API anahtarı oluşturdu?
- Kim workflow güncelledi?

Audit kayıtları uygulama loglarından ayrı tutulacaktır.

13. Fraud Detection Hooks

Platform, ileride gelişmiş sahtecilik analizlerini destekleyecek şekilde olay üretmelidir.

Örnek sinyaller:

- Olağandışı konum değişimleri
- Aynı hesabın kısa sürede çok farklı cihazlarda kullanılması
- Gerçekçi olmayan teslimat süreleri
- Tekrarlanan başarısız doğrulamalar

Bu sinyaller AI ve güvenlik servisleri tarafından değerlendirilebilir.

14. Encryption

Temel prensipler:

- Ağ trafiği TLS ile korunur.
- Hassas veriler uygun alanlarda şifrelenir.
- Parolalar tek yönlü güçlü algoritmalarla saklanır.
- Şifreleme anahtarları uygulamadan bağımsız yönetilir.

15. Observability for Security

Güvenlik yalnızca engellemek değildir.

Şüpheli davranışların görünür olması gerekir.

Bu nedenle:

- Güvenlik metrikleri
- Güvenlik logları
- Alarm kuralları
- Olay korelasyonu

başlangıçtan itibaren tasarlanacaktır.

16. Incident Response

Platform aşağıdaki olaylara hazırlıklı olmalıdır:

- Hesap ele geçirilmesi
- API anahtarı sızıntısı
- Kötü niyetli istemci
- Yetkisiz erişim denemeleri
- Hizmet engelleme girişimleri

Her olay için tanımlı müdahale prosedürü bulunacaktır.

17. Security Testing

Her sürümde aşağıdaki kontroller hedeflenecektir:

- Statik kod analizi
- Bağımlılık güvenlik taraması
- Birim ve entegrasyon testleri
- Yetkilendirme testleri
- API güvenlik testleri
- Yük testleri
- Penetrasyon testleri (yayın öncesi kritik sürümlerde)

18. Secure Defaults

Varsayılan ayarlar güvenli olacaktır.

Örneğin:

- Yeni kullanıcı minimum yetkiyle başlar.
- Yeni API anahtarları dar kapsamlı olur.
- Yeni tenant gereksiz özelliklerle gelmez.
- Geliştirme amaçlı kolaylaştırmalar üretimde kapalıdır.

19. Security Architecture Decision Records

Güvenlikle ilgili büyük kararlar ayrı ADR kayıtlarıyla belgelenir.

Örneğin:

- Neden JWT?
- Neden NATS?
- Neden PostgreSQL?
- Neden Kotlin/Swift?
- Neden Rust?

Böylece yıllar sonra bile teknik kararların gerekçesi kaybolmaz.

20. Security Success Criteria

Bir özellik güvenlik açısından tamamlanmış sayılabilmesi için:

- Kimlik doğrulama modeli tanımlanmış olmalı.
- Yetkilendirme kuralları belirlenmiş olmalı.
- Girdi doğrulamaları yapılmış olmalı.
- Hassas veriler uygun şekilde korunmuş olmalı.
- Audit kayıtları üretiliyor olmalı.
- Güvenlik testleri başarıyla geçmiş olmalı.

Mimaride değiştirmek istediğim bir nokta

Kardeşim, burada özellikle ilk taslaktan itibaren eklemek istediğim bir ilke var.

Ben **güvenliği yalnızca bir "Security Service"** olarak görmek istemiyorum.

Onun yerine güvenliği üç katmanda ele alalım:

1. Platform Security

Kimlik doğrulama, cihaz yönetimi, oturumlar, API güvenliği, gizli bilgiler.

2. Domain Security

İş kurallarına dayalı yetkilendirme. Örneğin "Bu kurye yalnızca kendi teslimatını tamamlayabilir."

3. Operational Security

Anomali tespiti, olay müdahalesi, denetim kayıtları, AI destekli risk analizi ve operasyonel görünürlük.

Bu ayrımın büyük avantajı şu olur:

- Güvenlik sadece giriş ekranında bitmez.
- İş kuralları güvenliğin doğal bir parçası olur.
- AI güvenlik katmanı ileride sisteme kolayca eklenebilir.
- Her servis kendi güvenlik sorumluluğunu bilir.

Bence bu yaklaşım, Qervon'u sadece güvenli bir uygulama değil, **güvenliği mimarisine işlemiş bir platform** haline getirir.

Sıradaki belge

Artık çekirdek mimari büyük ölçüde şekilleniyor.

Bence sıradaki doküman:

QAS-000005 — API & Integration Standard

Bu belge; REST kurallarını, OpenAPI sözleşmelerini, WebSocket mesaj formatlarını, hata kodlarını, sürümleme politikasını, idempotency kurallarını ve dış sistem entegrasyon standartlarını tanımlayacak. O belge tamamlandığında backend, web ve mobil ekipleri aynı teknik dili konuşmaya başlayabilecek.

Devam edelim kardeşim.

Harika kardeşim.

Şimdi yazacağımız belge, bana göre bütün backend ekibinin masasının üzerinde açık duracak belge.

Çünkü iyi yazılmış API, sadece istemciyle konuşmaz.

API;

- mimariyi korur,
- güvenliği korur,
- performansı korur,
- geleceği korur.

Bu yüzden bu belgeyi mümkün olduğunca **zamandan bağımsız** ve **teknolojiye bağlı olmayan kurallar** şeklinde yazıyorum.

QAS-000005

API & Integration Standard

Version: 0.1 Foundation Draft

1. Purpose

Bu belge;

Qervon Platformu içerisindeki bütün servislerin;

- REST API
- WebSocket
- Event Integration
- Internal Service Communication

standartlarını tanımlar.

Amaç;

bütün servislerin

tek dil konuşmasını sağlamaktır.

2. API Philosophy

API

bir Controller değildir.

API

Platform Contract'tır.

Bu nedenle

API

UI'dan bağımsızdır.

Mobil uygulama değişebilir.

Web değişebilir.

Ama API

mümkün olduğunca

değişmeyecektir.

3. Contract First Development

Bu maddeyi özellikle ekliyorum.

Qervon'da geliştirme sırası

şöyle olacaktır.

Business Requirement

↓

Domain Model

↓

OpenAPI Contract

↓

Review

↓

Tests

↓

Implementation

↓

Frontend

↓

Mobile
Hiçbir endpoint

önce kodlanmayacaktır.

4. API Versioning

Version

URL içinde olmayacaktır.

Yanlış

`/api/v1/orders`

Doğru yaklaşım:

- Stabil kaynak yolları (`/orders`, `/couriers` vb.)
- Sürüm bilgisinin HTTP başlığı (ör. **Accept**) veya API Gateway politikası üzerinden yönetilmesi.

Bunun avantajı:

- URL'ler yıllarca değişmez.
- Büyük sürüm geçişleri daha kontrollü yapılır.
- Gateway aynı anda birden fazla API sürümünü yönetebilir.

5. Resource Naming

Kaynaklar

isim olacaktır.

Fiil olmayacaktır.

Doğru

`/orders`

Yanlış

`/createOrder`

6. HTTP Method Rules

GET

okur.

POST

oluşturur.

PUT

tam değiştirir.

PATCH

kısmi değiştirir.

DELETE

siler.

7. Command Separation

İş kuralları

Controller içinde

olmayacaktır.

Controller yalnızca

isteği kabul eder.

8. Request Validation

Her istek

üç aşamada doğrulanacaktır.

Transport Validation

↓

Schema Validation

↓

Business Validation

Bu üçü birbirine karıştırılmayacaktır.

9. Response Standard

Bütün başarılı cevaplar

aynı yapıya sahip olacaktır.

Örnek yapı:

```
{  
  "data": {},  
  "meta": {},  
  "links": {}  
}
```

10. Error Standard

Hata mesajları

insan okuyabilmesi için değil,

istemcinin işleyebilmesi için tasarlanacaktır.

Örnek:

```
{  
  "error": {  
    "code": "ORDER_NOT_FOUND",  
    "message": "Order not found."  
  }  
}
```

Yerelleştirme istemci tarafında yapılabilir.

11. Correlation ID

Her HTTP isteği

Correlation ID taşıyacaktır.

Bu ID

loglara

event'lere

trace'lere

aktarılacaktır.

12. Idempotency

Aynı POST isteği

iki kez gönderildiğinde

iki sipariş oluşmamalıdır.

Bu nedenle

kritik endpoint'ler

Idempotency-Key

destekleyecektir.

13. Pagination

Sayfalama

tek standart kullanacaktır.

Örnek:

`limit`

`cursor`

Offset tabanlı sayfalama yalnızca uygun olduğu yerlerde kullanılacaktır.

Cursor tabanlı yaklaşım büyük veri kümelerinde tercih edilecektir.

14. Filtering

Filtreleme

tek standartta olacaktır.

Örneğin:

`status=ACTIVE`

`created_after=...`

`created_before=...`

`tenant=...`

15. Sorting

Tek format.

Örneğin

`sort=createdAt`

`sort=-createdAt`
eksi

descending

anlamına gelir.

16. Sparse Fieldsets

İstemci

gereksiz veri istemeyecektir.

Örneğin

`fields=id,name,status`

Bu özellikle mobil istemcilerde veri miktarını azaltır.

17. Expansion

İlişkili veriler

otomatik gelmeyecektir.

Örneğin

`expand=courier`
veya

`expand=customer`

İstemci gerçekten ihtiyaç duyduğu ilişkileri ister.

18. WebSocket Standard

WebSocket

yalnızca

gerçek zamanlı olaylar içindir.

CRUD

yapılmayacaktır.

Mesajlar

tek format kullanacaktır.

```
{  
  "type": "...",  
  "event": "...",  
  "payload": {}  
}
```

19. API Security

Her endpoint

varsayılan olarak

kimlik doğrulaması gerektirir.

Anonim endpoint'ler

özellikle işaretlenmelidir.

20. Integration Principles

Harici sistemler

doğrudan

Order Service'e

bağlanmaz.

API Gateway

veya

Integration Layer

üzerinden bağlanırlar.

21. Retry Policy

Retry

yalnızca

güvenli endpoint'lerde yapılacaktır.

Örneğin

GET

retry edilebilir.

POST

ancak

idempotency desteği varsa

retry edilir.

22. Rate Limiting

Sistem farklı seviyelerde sınır koyabilir.

Örneğin:

- IP
- Kullanıcı
- Tenant
- API anahtarı
- Endpoint

Bu politika API Gateway tarafından uygulanabilir.

23. Deprecation Policy

Bir endpoint

aniden kaldırılmaz.

Önce:

Deprecated

durumuna alınır.

Dokümanite edilir.

Geçiş süresi tanınır.

Sonra kaldırılır.

24. API Documentation

Her endpoint

OpenAPI üzerinden

otomatik dokümanite edilir.

Dokümantasyon

kaynak koddan ayrı tutulmaz.

Tek doğruluk kaynağı

OpenAPI sözleşmesidir.

25. SDK Generation

OpenAPI sözleşmesinden otomatik olarak:

- Kotlin istemcisi
- Swift istemcisi
- TypeScript istemcisi
- Rust istemcisi

üretilcektir.

Böylece istemci ve sunucu arasında model farklılıkları en aza indirilir.

Burada değiştirmek istediğim çok önemli bir nokta

Kardeşim, ilk taslakta API sürümlemesi için "URL yerine header kullanalım" önerisini yazmıştım.

Ben bunu biraz daha olgunlaştırmak istiyorum.

Bence Qervon'da **üç seviyeli uyumluluk politikası** olmalı:

- 1. API Compatibility**
Mümkün olduğunca geriye dönük uyumluluk korunur.
- 2. Contract Compatibility**
OpenAPI sözleşmesi kırılmadan istemciler etkilenmez.
- 3. Event Compatibility**
Event şemaları ayrı sürümlenir ve eski tüketiciler çalışmaya devam edebilir.

Bu üç katman birbirinden bağımsız yönetildiğinde, web, Android, iOS ve dış entegrasyonlar farklı hızlarda güncellenebilse bile platform uzun yıllar kararlı kalabilir.

Küçük ama önemli bir revizyon önerisi

URL'de sürüm numarası kullanmama fikri güçlü bir seçenek olsa da, bazı kurumlar ve üçüncü taraf entegrasyonları için URL tabanlı sürümleme daha pratik olabilir. Bu nedenle bunu kesin bir kural yerine **mimari kararlar seçilen politika** olarak tanımlamayı öneriyorum:

- İç servisler ve modern istemciler için header veya gateway tabanlı sürümleme tercih edilebilir.
- Dış entegrasyonlar için gerektiğinde URL tabanlı sürümleme (`/api/v1/...`) desteklenebilir.

Bu esneklik, Qervon'un hem modern hem de kurumsal entegrasyon senaryolarında daha rahat kullanılmasını sağlar.

Sıradaki belge

Artık çekirdek servislerin konuşma dili oluştu.

Bence bundan sonra yazmamız gereken belge:

QAS-000006 — Database & Persistence Standard

Bu belge yalnızca tablo isimlerini anlatmayacak; veri modelleme ilkelerini, PostgreSQL ve PostGIS kullanımını, indeksleme stratejilerini, migration kurallarını, partitioning, arşivleme, veri saklama politikalarını ve Event Store ile operasyonel veritabanı arasındaki ilişkiyi tanımlayacak.

Benim gözümde bu belge, Rust servislerinin omurgasını oluşturacak en kritik teknik standartlardan biri olacak.

Devam edelim kardeşim.

Harika kardeşim.

Şimdi yazacağımız belgeyi ben **Qervon'un omurgası** olarak görüyorum.

Çünkü bugün dünyadaki birçok sistem, veritabanı kötü tasarlandığı için yıllar sonra yeniden yazılıyor.

Ben Qervon'un 15–20 yıl sonra bile aynı temel üzerinde büyüebilmesini istiyorum.

Bu nedenle bu belgeyi sadece "PostgreSQL kullanacağız" seviyesinde değil, **veri mühendisliği standardı** olarak yazacağız.

QAS-000006

Database & Persistence Standard

Version: 0.1 Foundation Draft

1. Purpose

Bu belge;

Qervon Platformu içerisindeki tüm kalıcı veri yönetimini tanımlar.

Kapsam:

- PostgreSQL
- PostGIS
- Redis
- Object Storage
- Event Store
- Search Index
- Cache
- Snapshot
- Archive

2. Persistence Philosophy

Golden Rule

Veri sonsuza kadar yaşayabilir. Kod yaşayamaz.

Bu nedenle

veri modeli,

koddan daha fazla özenle tasarlanacaktır.

Database is not Business Logic

Veritabanı

iş kurallarını taşımaz.

İş kuralları

Domain Layer'dadır.

Database yalnızca

veriyi saklar.

3. Polyglot Persistence

Qervon tek veritabanına bağlı değildir.

Her teknoloji

kendi amacı için kullanılacaktır.

Teknoloji	Amaç
PostgreSQL	Operasyonel veri
PostGIS	Coğrafi veriler
Redis	Cache ve geçici durum
Object Storage	Dosyalar
Event Store	Olay geçmişi
Search Index	Tam metin arama

Hiçbir teknoloji başka birinin görevini üstlenmez.

4. PostgreSQL

PostgreSQL

platformun ana veritabanıdır.

Tercih sebepleri:

- ACID
- MVCC
- JSONB
- PostGIS
- Partitioning
- Logical Replication
- Maturity
- Rust ekosistemiyle güçlü uyum

5. Database Ownership

Bu madde değiştirilemez.

Her servis

kendi verisinin sahibidir.

Order Service

|

orders

order_items

order_history

Dispatch Service

gidip

orders

tablosunu

değiştiremez.

6. No Cross Database Writes

Hiçbir servis

başka servisin

tablosuna

INSERT

UPDATE

DELETE

yapamaz.

İletişim

yalnızca

Event

veya

API

ile olur.

7. Naming Convention

Table

snake_case

order_items

Column

snake_case

created_at

Primary Key

Her tabloda

tek tip olacaktır.

id

Foreign Key

order_id

Timestamp

Her tabloda

zorunludur.

created_at

updated_at

Soft Delete

Fiziksel silme varsayılan davranış değildir.

Silme gerekiyorsa:

deleted_at
alanı kullanılır.

Ancak burada bir istisna tanımlıyorum:

KVKK/GDPR kapsamında gerçekten silinmesi gereken kişisel veriler, ilgili veri saklama politikasına göre anonimleştirilebilir veya fiziksel olarak silinebilir.

8. UUID Strategy

Burada çok önemli bir karar.

Ben klasik

UUID v4

kullanmayacağım.

Önerim:

UUID v7

Neden?

- Zaman sıralı
- İndeks dostu
- Daha iyi yazma performansı
- Dağıtık sistemler için uygun

Tüm servislerde tek kimlik standardı kullanılacaktır.

9. Optimistic Locking

Her Aggregate

aşağıdaki alanı taşıyacaktır.

`version`

Bu sayede

eşzamanlı güncellemeler

güvenli şekilde yönetilir.

10. Audit Fields

Her tabloda

zorunlu alanlar.

`created_at`

`updated_at`

`created_by`

updated_by
Opsiyonel:

deleted_at

deleted_by

11. JSON Usage

JSONB

yalnızca

esnek

metadata

için kullanılacaktır.

Ana iş verileri

JSON içinde saklanmayacaktır.

12. Index Policy

Her indeksin

bir gerekçesi olacaktır.

Fazla indeks

yasaktır.

İndeks

ölçülerek eklenir.

13. Partition Strategy

Büyük tablolar

partition edilecektir.

Örneğin

events

↓

2026

↓

2027

↓

2028

veya

tenant_id

bazlı.

14. Archiving

Operasyonel veritabanı

sonsuz büyümeyecektir.

Eski veriler

Archive Database'e

taşınabilir.

Arşiv politikaları yasal yükümlülükler ve iş ihtiyaçlarına göre tanımlanacaktır.

15. Read Model

CQRS kullanılan alanlarda

Read Database

ayrı olabilir.

Operasyonel

yazma

veritabanı

ile

okuma modeli

birbirinden ayrılabilir.

16. Cache Strategy

Redis

gerçeğin sahibi değildir.

Redis

her zaman

yeniden oluşturulabilir.

17. Object Storage

Fotoğraf

PDF

İmza

Video

Ses

dosyaları

Object Storage'da tutulacaktır.

Veritabanında yalnızca

referans bulunacaktır.

18. Search

Arama

SQL ile yapılmayacaktır.

Search Service

kullanılacaktır.

İlk sürümde

Meilisearch

veya

OpenSearch

değerlendirilecektir.

19. Backup Strategy

Yedekleme

tek tip olmayacaktır.

- Tam yedek
- Artımlı yedek
- Noktaya dönüş (Point-in-Time Recovery)
- Coğrafi olarak ayrı kopyalar

hedeflenecektir.

Yedekler düzenli olarak geri yükleme testinden geçirilmelidir.

20. Migration Policy

Elle SQL çalıştırmak

yasaktır.

Her değişiklik

Migration'dır.

Migration

geri alınabilir

olmalıdır.

Üretim veritabanında manuel şema değişikliği yapılmayacaktır.

21. Geospatial Standard

PostGIS

platformun önemli parçalarından biridir.

Konum;

latitude

longitude

şeklinde iki sayı olarak değil,

coğrafi tiplerle yönetilecektir.

Desteklenecek örnek işlemler:

- Yakındaki kuryeler
- Bölge içinde mi?
- Rota uzunluğu
- Geofence kontrolü
- Servis alanı

22. Data Retention

Her veri

aynı süre tutulmayacaktır.

Örnek kategoriler:

Veri	Politika
Audit	Uzun süre
Finans	Yasal zorunluluklara göre
Konum geçmişi	Tanımlı süre
Cache	Kısa süre
Geçici dosyalar	Otomatik temizleme

23. Data Classification

Her tablo

sınıflandırılacaktır.

Örneğin:

PUBLIC

INTERNAL

CONFIDENTIAL

RESTRICTED

Bu sınıf;

yedekleme,
şifreleme,
erişim,
saklama
politikalarını etkiler.

24. Performance Budget

Her tablo

performans hedeflerine sahip olmalıdır.

Örneğin:

- Kritik sorgular
- Maksimum beklenen satır sayısı
- İndeks stratejisi
- Beklenen büyüme

dokümanite edilir.

25. Schema Evolution

Şema değişiklikleri

aşağıdaki sırayla yapılacaktır.

Yeni alan ekle

↓

Kod yeni alanı desteklesin

↓

Eski alan kullanım dışı kalsın

↓

Veri taşınsın

↓

Eski alan kaldırılabilsin

Doğrudan

alan silmek

yasaktır.

Burada değiştirmek istediğim çok önemli konu

Kardeşim, önceki taslaklarda **her servisin kendi veritabanı** olacak yaklaşımını kullandık.

Bunu biraz daha netleştirmek istiyorum.

Benim önerim şu:

- **Her servisin kendi veri sahipliği** olacak. Bu değişmeyecek.
- Fiziksel olarak ise başlangıçta **tek PostgreSQL kümesi** içinde, servis bazında ayrı şemalar (`orders`, `dispatch`, `identity` gibi) kullanabiliriz.
- Trafik arttığında, mimari değişmeden bu şemaları ayrı veritabanlarına veya ayrı PostgreSQL kümelerine taşıyabiliriz.

Bu yaklaşımın avantajı:

- İlk sürüm daha yönetilebilir olur.
- Mikroservis prensibi bozulmaz.
- Operasyonel maliyet düşük kalır.
- Büyüdüğümüzde yeniden yazmaya gerek kalmadan ölçeklenebiliriz.

Yani veri **sahipliği** ile fiziksel **yerleşimi** birbirine karıştırmayacağız. Bu ayrım, uzun ömürlü sistemlerde çok büyük fark yaratır.

Bir sonraki belge

Artık çekirdeğin büyük kısmını tanımladık.

Bence sıradaki doküman:

QAS-000007 — Mobile Platform Architecture

Bu belge Android (Kotlin) ve iOS (Swift) mimarisini, ortak API sözleşmelerini, offline-first yaklaşımını, arka plan görevlerini, konum takibini, güvenlik katmanlarını, bildirim akışını ve mağaza dağıtım stratejilerini ayrıntılı olarak tanımlayacak.

Benim görüşüme göre bu belge, kurye uygulamasının güvenilirliğini belirleyen en kritik teknik referanslardan biri olacak.

Devam edelim kardeşim.

Harika kardeşim.

Şimdi gerçekten projenin en kritik belgelerinden birine geldik.

Çünkü Qervon'un başarısını belirleyecek olan Backend değil.

Mobil uygulamalar olacak.

Kurye, günde 8-12 saat bu uygulamaya bakacak.

Bir butonun yeri bile operasyonu etkileyebilir.

Bu yüzden bu belgeyi sıradan bir "Android Architecture" dokümanı olarak yazmayacağız.

Bu belge;

Google Android Engineering

-

Apple Human Interface Guidelines

-

Material Design

-

Enterprise Mobility

-

Field Workforce Applications

seviyesinde olacak.

QAS-000007

Mobile Platform Architecture

Version 0.1 Foundation Draft

1. Purpose

Bu belge

Android

ve

iOS

uygulamalarının

mimarisini tanımlar.

Qervon

iki ayrı native uygulama geliştirecektir.

Android

↓

Kotlin

iOS

↓

Swift

2. Mobile Philosophy

Mobil uygulama

iş kurallarının sahibi değildir.

Mobil uygulama

yalnızca

kullanıcı deneyiminden

sorumludur.

İş kuralları

Rust Backend'dedir.

3. Native First

Çok önemli karar.

Flutter

yok.

React Native

yok.

Kotlin

ve

Swift

kullanılacaktır.

Sebepleri

- maksimum performans
- minimum gecikme
- tam platform desteği
- güvenlik
- background task
- kamera
- bluetooth
- NFC
- biometrics
- GPS

tam destek.

4. Offline First

En önemli madde.

Kurye

internetsiz çalışabilir.

Örneğin

Teslimat

↓

Fotoğraf

↓

İmza

↓

GPS

↓

QR

↓

PIN

İnternet yok.

Sorun değil.

Her işlem

yerel veritabanına

kaydedilir.

Bağlantı gelince

otomatik senkronize edilir.

5. Synchronization Engine

Mobil uygulamanın

en önemli modülü.

SQLite / Room / SwiftData

↓

Sync Queue

↓

Conflict Resolver

↓

Retry Engine

↓

Rust Backend

6. Conflict Resolution

İki cihaz

aynı kaydı

değiştirirse

çakışma oluşabilir.

Kurallar:

- Server Wins
- Client Wins
- Merge
- Manual Review

Her Entity

hangi stratejiyi kullandığını

belirtmelidir.

7. Background Processing

Android

WorkManager

Foreground Service

gerektiğinde kullanılır.

iOS

BackgroundTasks

ve uygun sistem mekanizmaları kullanılır.

Arka plan işlemleri

işletim sistemi kurallarına uygun olacaktır.

8. Local Database

Android

Room

iOS

SwiftData veya Core Data

Yerel veritabanı

şifrelenebilir olacak.

9. Secure Storage

Hassas bilgiler

yerel veritabanında tutulmaz.

Android

↓

Keystore

iOS

↓

Keychain

10. Authentication

Desteklenecek yöntemler

- OTP
- Password
- Biometric
- Passkey (gelecekte)

Çok faktörlü kimlik doğrulama

desteklenecektir.

11. Push Notifications

Push

yalnızca

bildirim değildir.

Push

iş akışının

parçasıdır.

Örneğin

Yeni Görev

↓

Push

↓

Kurye Açtı

↓

Ack

↓

Backend

Her kritik push

teslim edildi mi

okundu mu

takip edilir.

12. Location Engine

Qervon'un

en karmaşık modüllerinden biri.

Destekler:

- Continuous Tracking

- Significant Change
- Geofence
- Adaptive Accuracy
- Battery Aware Tracking

Konum

her saniye

gönderilmeyecektir.

Akıllı algoritmalar

kullanılacaktır.

13. Map Engine

Harita

soyut katman olacaktır.

İlk sürümde:

MapLibre + OpenStreetMap

İhtiyaca göre:

Google Maps

Apple MapKit

HERE

TomTom

eklenebilir.

14. Camera Architecture

Kamera

soyut servis olacaktır.

Destekler:

- QR

- Barcode
- OCR
- Document Scan
- Signature Capture
- Photo Evidence

15. File Upload

Dosyalar

önce

yerel kuyruğa alınır.

Sonra

arka planda

yüklenir.

Bağlantı koparsa

yükleme devam ettirilebilir.

16. UI Architecture

Android

MVVM

-

Clean Architecture

iOS

MVVM

-

Coordinator

veya uygun yönlendirme deseni.

UI

iş mantığı içermez.

17. Navigation

Navigasyon

merkezi yönetilecektir.

Deep Link

desteklenecektir.

18. Feature Modules

Her özellik

ayrı modül olacaktır.

Örneğin

Login

Orders

Tracking

Maps

Payments

Settings

Profile

Birbirlerinden bağımsız geliştirilebilir.

19. Telemetry

Mobil uygulama

şunları üretecektir.

- Crash
- Performance
- Memory

- Battery
- Network
- Sync
- GPS

Bu veriler

kişisel verileri gereksiz yere toplamadan

operasyonel iyileştirme amacıyla kullanılacaktır.

20. Accessibility

Bu maddeyi özellikle ekliyorum.

Qervon

WCAG

uyumlu olacaktır.

Destek:

- Screen Reader
- Dynamic Font
- High Contrast
- Color Blind Safe
- Keyboard Navigation (uygun platformlarda)

21. Mobile Design System

Android

Material 3

iOS

Apple Human Interface Guidelines

Kendi platformlarının davranışlarını koruyacaktır.

Android'i iPhone gibi,

iPhone'u Android gibi
yapmayacağız.

22. Update Strategy

Android

Play Store

ve gerektiğinde kurumsal dağıtım.

iOS

TestFlight

↓

App Store

Güncellemeler

geriye dönük uyumluluk gözetilerek yayımlanacaktır.

23. Crash Recovery

Uygulama

çökerse

iş kaybolmayacaktır.

Restart

↓

Sync Queue

↓

Devam

24. Battery Budget

En önemli maddelerden biri.

Kurye uygulaması

bataryayı tüketen uygulama

olmayacaktır.

GPS

akıllı yönetilecektir.

25. Mobile Security Review Checklist

Her sürümde aşağıdaki maddeler gözden geçirilecektir:

- Hassas veriler güvenli saklanıyor mu?
- Sertifika doğrulaması doğru çalışıyor mu?
- Root/Jailbreak kontrolleri beklendiği gibi mi?
- Yetkisiz ekranlara erişim engelleniyor mu?
- Offline kuyruk güvenli mi?
- Günlükler hassas veri içermiyor mu?

Şimdi çok önemli bir mimari kararı öneriyorum

Kardeşim, burada önceki konuşmalarımızdan farklı bir önerim var.

Ben Android ve iOS uygulamalarını **özellik (feature) bazlı** geliştirmek istiyorum.

Yani:

```
mobile/  
  android/  
    features/  
      login/  
      orders/  
      tracking/  
      maps/  
      payments/
```

ve

```
mobile/  
  ios/  
    Features/  
      Login/  
      Orders/  
      Tracking/  
      Maps/  
      Payments/
```

Bu yaklaşımın avantajları:

- Her özellik bağımsız geliştirilebilir.
- Test kapsamı daha net olur.
- Yeni geliştiriciler projeye daha kolay adapte olur.
- Büyük ekiplerde çakışmalar azalır.
- Aynı iş akışı Android ve iOS'ta paralel ilerleyebilir.

Burada düzeltmek istediğim bir nokta

Önceki tasarlarda iOS için yalnızca SwiftData demiştik.

Bence bunu kesin bir teknoloji kararı olarak yazmayalım.

Onun yerine:

- Varsayılan tercih: **SwiftData** (uygun minimum iOS sürümü destekliorsa)
- Geriye dönük uyumluluk veya özel gereksinimler varsa: **Core Data**

şeklinde tanımlayalım.

Böylece mimari belge uzun ömürlü olur; belirli bir Apple sürümüne gereksiz yere bağımlı kalmaz.

Sıradaki belge

Bence artık web tarafına geçebiliriz:

QAS-000008 — Web Platform Architecture

Bu belge sadece Admin Paneli ve Müşteri PWA'yı anlatmayacak; aynı zamanda tasarım sistemi, bileşen mimarisi, erişilebilirlik, performans, çoklu dil desteği, tema sistemi, gerçek zamanlı veri akışı ve offline PWA davranışlarını da tanımlayacak.

Benim hedefim, Qervon'un web tarafını da mobil kadar disiplinli ve uzun ömürlü bir mimariyle inşa etmek.

Devam edelim kardeşim.

Harika kardeşim.

Şimdi artık **Qervon'un vitrinini** yazıyoruz.

Ama ben Admin Panel'e "web sitesi" gibi bakmıyorum.

Ben onu;

Mission Control Center

olarak görüyorum.

NASA'nın kontrol merkezi gibi...

FedEx'in operasyon merkezi gibi...

Amazon Logistics'in kontrol ekranı gibi...

Yani yöneticiler bütün şirketi tek ekrandan yönetecek.

Dolayısıyla bu belge sadece React mimarisini değil, **operasyon deneyimini** de tanımlayacak.

QAS-000008

Web Platform Architecture

Version: 0.1 Foundation Draft

1. Purpose

Bu belge;

Qervon Web Platformu'nun mimarisini tanımlar.

Kapsam:

- Admin Portal
- Customer Portal
- Customer PWA
- Shared Component Library
- Design System

2. Vision

Web Platform;

CRUD ekranlarından oluşan

bir panel değildir.

Web Platform;

gerçek zamanlı

operasyon merkezidir.

Kullanıcı

aynı anda

- siparişleri,
- kuryeleri,
- haritayı,
- finansı,
- uyarıları,
- AI önerilerini

izleyebilmelidir.

3. Technology Stack

Frontend

React

TypeScript

Vite

Tailwind CSS

TanStack Query

Zustand

React Router

PWA

MapLibre

WebSocket

4. Architectural Style

Web

Feature First

geliştirilecektir.

Yanlış

components/

pages/

hooks/

utils/

Bu yapı küçük projeler için uygundur.

Doğru

features/

orders/

dispatch/

fleet/

finance/

analytics/

notifications/

shared/

core/

design-system/

Her özellik

kendi içinde

tam bağımsız olacaktır.

5. Feature Architecture

Örnek

orders/

api/

components/

hooks/

pages/

models/

routes/

store/

tests/

styles/

Her feature

kendi yaşam döngüsüne sahip olacaktır.

6. Shared Layer

Shared

herkesin kullandığı

kod değildir.

Shared

yalnızca

gerçekten ortak olan

kod içindir.

Örneğin

Button

Modal

Table

Dialog

Toast

Icon

Avatar

7. Core Layer

Core

uygulamanın

altyapısıdır.

Örneğin

Authentication

Localization

Configuration

Routing

Theme

Logger

API Client

Error Handler

Permissions

8. Design System

Qervon

kendi

Design System'ine

sahip olacaktır.

Bu sistem

Material UI

değildir.

Ant Design

değildir.

Bizim olacaktır.

İçinde

Buttons

Typography

Spacing

Colors

Icons

Elevation

Motion

Animation

Grid

Responsive Rules
bulunacaktır.

9. Component Rules

Her component

tek sorumluluğa sahip olacaktır.

Component

iş kuralı içermez.

10. Smart / Dumb Components

Akıllı component

veri alır.

Sunum component'i

yalnızca gösterir.

11. State Management

Burada çok önemli karar.

Global State

yalnızca

gerçekten global

olan şeyler için.

Örneğin

Theme

Language

Current User

Permissions

Configuration

Sipariş listesi

global state

olmayacaktır.

TanStack Query ile sunucu durumu yönetilecek, istemciye özgü UI durumu ise küçük ve izole store'larda tutulacaktır.

12. API Layer

API çağrıları

component içinde

olmayacaktır.

Component

↓

Hook

↓

API Client



Rust Backend

13. Realtime

Gerçek zamanlı veri

WebSocket

üzerinden gelecektir.

Örneğin

Courier Location



WebSocket



Store



Map

14. Forms

Her form

aynı standartta olacaktır.

Destek

- Validation
- Autosave
- Draft
- Undo
- Dirty State

15. Tables

Qervon'da

tablolar

çok önemlidir.

Standart özellikler

- Pagination
- Virtual Scroll
- Sticky Header
- Column Resize
- Column Hide
- Export
- Search
- Filters
- Saved Views

16. Map Components

Harita

component değildir.

Framework'tür.

Harita

üzerine

özellikler eklenir.

Örneğin

Courier Layer

Order Layer

Traffic Layer

Heat Map

Cluster

Geofence

Drawing

17. Dashboard Engine

Dashboard

hardcoded olmayacaktır.

Widget tabanlı olacaktır.

Örneğin

Revenue Widget

Map Widget

Courier Widget

AI Widget

Weather Widget

Admin

bunları

sürükleyip bırakabilir.

18. Theme System

Desteklenecek

- Light
- Dark
- High Contrast

Kurumsal müşteriler için

tenant bazlı

renk temaları

tanımlanabilir.

19. Localization

Çok dilli

başlangıçtan itibaren desteklenecektir.

Çeviriler

anahtar tabanlı olacaktır.

Metinler

koda gömülmeyecektir.

20. Accessibility

WCAG

AA

hedeflenecektir.

Destek:

- Screen Reader
- Keyboard Navigation
- Focus Management
- High Contrast
- Reduced Motion

21. Performance Budget

İlk yükleme

olabildiğince küçük tutulacaktır.

Yöntemler:

- Route splitting
- Lazy loading
- Code splitting
- Asset optimization

- Görsel önbellekleme

Performans hedefleri ölçülerek takip edilecektir.

22. PWA Strategy

Customer Portal

aynı zamanda

PWA olacaktır.

Destek:

- Install
- Offline Cache
- Push
- Background Sync (desteklenen platformlarda)
- App Shortcut

23. Error Boundaries

Bir component

çökerse

uygulama

çökmeyecektir.

Error Boundary

kullanılacaktır.

24. Observability

Frontend

şunları raporlayacaktır.

- Crash
- API Errors
- Performance

- WebSocket Status
- User Journey

Kişisel veriler gereksiz yere toplanmayacak ve gizlilik politikalarına uygun hareket edilecektir.

25. Micro Frontend Policy

Şu anda

Micro Frontend

kullanılmayacaktır.

Sebep:

Erken optimizasyon

karmaşıklık getirir.

Ancak mimari

ileride destekleyecek şekilde

tasarlanacaktır.

26. Command Center (Yeni Bölüm)

Bu bölümü özellikle ekliyorum.

Admin Panell'in ana ekranı klasik dashboard olmayacak.

Qervon Command Center olacak.

Ana ekran:

```
+-----+
+
| Global Search | AI Assistant | Alerts | User | Settings
|
+-----+
+
| KPI | Active Couriers | Live Orders | SLA | Weather | API
|
+-----+
+
|
|
```

Yönetici, sisteme girdiğinde ilk gördüğü ekran rapor değil, **operasyonun canlı resmi** olacak.

27. AI Workspace

Bence burada çok büyük bir fark oluşturabiliriz.

Admin Panel'e yerleşik bir **AI Workspace** ekleyelim.

Yönetici doğal dille yazabilecek:

"Son 7 günde en çok geciken bölge hangisi?"

veya

"Bugün hangi kuryelerin iş yükü fazla?"

AI, SQL yazmadan, rapor oluşturmada bu sorulara cevap verecek.

İleride bu alan;

- rapor üretme,
- operasyon önerileri,
- tahminleme,
- anomali analizi,
- karar destek sistemi

olarak genişleyebilir.

Şimdi çok önemli bir mimari kararı değiştiriyorum

Kardeşim, önceki taslaklarda **Admin Panel** ve **Customer Portal**'ı iki ayrı React uygulaması gibi düşünmüştük.

Bence daha doğru yaklaşım şu:

- **Tek Web Platform**

- Ortak tasarım sistemi
- Ortak altyapı
- Ortak API istemcisi
- Ortak kimlik doğrulama
- Ortak bileşen kütüphanesi

Bunun üzerinde:

- Admin Workspace
- Customer Workspace

ayrı çalışma alanları (workspace) olarak çalışsın.

Bu yaklaşım sayesinde:

- kod tekrarını azaltırız,
- bakım maliyetini düşürürüz,
- kullanıcı deneyimini tutarlı hale getiririz,
- yeni modülleri çok daha hızlı geliştirebiliriz.

Bir sonraki belge

Artık platformun temel katmanlarını tanımladık.

Bence sıradaki en önemli doküman:

QAS-000009 — AI Architecture Specification

Ama burada klasik "chatbot" yazmayacağız.

Benim hedefim, AI'yı Qervon'un **ayrı bir modülü değil, platformun doğal bir yeteneği (platform capability)** haline getirmek.

Yani AI; Dispatch, Analytics, Fraud Detection, ETA, Workflow, Raporlama ve Operasyon Merkezi'nin tamamına nüfuz eden ortak bir zekâ katmanı olacak. Bence Qervon'u piyasadaki benzerlerinden ayıracak en güçlü teknik avantaj da bu olacak.

Devam edelim kardeşim.

Harika kardeşim.

Bence artık Qervon'u rakiplerinden ayıracak yere geldik.

Çünkü bugün piyasadaki ürünlerin çoğunda "AI" diye eklenmiş bir sohbet kutusu var.

Ben bunu istemiyorum.

Ben AI'nın **ürünün kendisi** olmasını istiyorum.

Yani kullanıcı AI'yı kullanmayacak.

Platform zaten AI ile çalışacak.

İşte Qervon'un dünyadaki farkı burada başlayacak.

QAS-000009

Artificial Intelligence Architecture Specification

Version: 0.1 Foundation Draft

1. Purpose

Bu belge,

Qervon Platformu içerisindeki

bütün yapay zekâ

altyapısını tanımlar.

AI

ayrı ürün değildir.

AI

Platform Capability'dir.

2. Vision

Qervon AI

ChatGPT

değildir.

Qervon AI

Copilot

değildir.

Qervon AI

işletim sisteminin

karar destek katmanıdır.

3. AI Philosophy

En önemli ilke.

AI

karar vermez.

AI

öneri üretir.

Kararı

iş kuralları

veya

yetkili kullanıcı

verir.

Bu yaklaşım, özellikle finansal işlemler, teslimat iptalleri veya personel yönetimi gibi yüksek etkili alanlarda güvenliği ve hesap verebilirliği korur.

4. AI Layers

Qervon AI

tek model değildir.

Presentation AI

↓

Workflow AI

↓

Analytics AI

↓

Prediction AI

↓

Optimization AI

↓

Reasoning AI

↓

Infrastructure AI

Her katmanın

görevi farklıdır.

5. AI Gateway

Çok önemli karar.

Hiçbir servis

LLM

çağırmayacaktır.

Her şey

AI Gateway

üzerinden geçecektir.

Order Service

↓

AI Gateway

↓

LLM

↓

Response

Bu sayede:

- model değişebilir,

- sağlayıcı değişebilir,
- maliyet kontrol edilir,
- güvenlik denetlenir,
- gözlemlenebilirlik sağlanır.

6. Multi Model Strategy

Tek modele bağımlı olmayacağız.

AI Gateway aşağıdaki türde sağlayıcıları destekleyecek şekilde tasarlanacaktır:

- Büyük dil modelleri
- Küçük yerel modeller
- Görüntü modelleri
- Konuşma modelleri
- Gömme (Embedding) modelleri

Model seçimi görev tipine göre yapılacaktır.

7. Capability Based AI

En önemli tasarım.

AI

Prompt

çalıştırmaz.

AI

Capability

çalıştırır.

Örneğin

EstimateETA

AssignCourier

PredictDemand

GenerateReport

DetectFraud

SummarizeRoute

OptimizeFleet
Her capability

ayrı modüldür.

8. AI Context Engine

LLM

ham veritabanına

erişmez.

AI

önce

Context Engine'den

bağlam alır.

User

↓

Context Engine

↓

Relevant Data

↓

LLM

Bu sayede gereksiz veri paylaşılmaz.

9. Retrieval Strategy

AI

rastgele

veri okumaz.

Veri

aşağıdaki kaynaklardan gelir.

- API
- Event Stream
- Read Model
- Vector Index (gerektiğinde)
- Knowledge Base

10. Prompt Registry

Prompt

koda gömülmeyecektir.

Her Prompt

versiyonlanacaktır.

Örneğin

PROMPT-001

Dispatch Optimization

v3

Prompt'lar da yazılım bileşeni gibi test edilir.

11. Tool Calling

AI

doğrudan

SQL

çalıştırmayacaktır.

AI

yalnızca

tanımlı Tool'ları

çağıracaktır.

Örneğin

SearchOrders ()

CalculateETA ()

AssignCourier()

GenerateInvoice()

12. AI Memory

Burada önemli ayrım.

AI

insan gibi

hafıza tutmaz.

AI

yalnızca

izin verilen

iş bağlamını

kullanır.

Kullanıcıya ait kişisel bilgiler, veri sınıflandırması ve saklama politikalarına uygun şekilde işlenir.

13. AI Safety

Her AI cevabı

filtrelenir.

Kontroller:

- Yetki
- Veri sınıflandırması
- Gizlilik
- Hassas bilgi
- Politika uyumu

14. Human Approval

Bazı işlemler

onaysız yapılamaz.

Örneğin

Refund

↓

AI Suggestion

↓

Manager Approval
veya

Bulk Courier Assignment

↓

AI Plan

↓

Dispatcher Approval

15. AI Audit

AI

ne yaptı

neden yaptı

hangi modele sordu

hangi bağlamı kullandı

kaydedilir.

16. Explainability

AI

önerisini

açıklayabilmelidir.

Örneğin

Bu kurye seçildi çünkü:

- Mesafe uygun
- Trafik düşük

- Araç tipi uygun
- Son 2 saat içinde boş kapasitesi var

Yani sonuç değil,

gerekçe de üretilecektir.

17. AI Cost Control

AI

en pahalı modelle

çalışmayacaktır.

Gateway

göreve göre

uygun modeli seçebilir.

Örneğin:

- Basit sınıflandırma → küçük model
- Karmaşık rapor → büyük model
- ETA tahmini → özel algoritma

18. AI Observability

Her AI isteği için:

- Gecikme
- Maliyet
- Model
- Başarı durumu
- Token kullanımı
- Hata nedeni

izlenebilir olacaktır.

19. AI Failover

AI

çalışmazsa

platform

çalışmaya devam eder.

Örneğin

AI ETA

↓

Unavailable

↓

Rule Based ETA
veya

AI Dispatch

↓

Unavailable

↓

Rule Engine Dispatch
AI

zorunlu bağımlılık olmayacaktır.

20. AI Governance

AI davranışı

sürüm kontrollü olacaktır.

Yeni model

hemen

canlıya alınmaz.

A/B testleri,

gözlem,

gerekirse geri dönüş

planları uygulanır.

21. AI Plugin Architecture

Qervon AI

plugin destekleyecektir.

Örneğin

Dispatch AI

Fraud AI

Analytics AI

Forecast AI

Warehouse AI

Medical AI

Her biri

bağımsız geliştirilebilir.

22. AI Agents (Revize Edilmiş Yaklaşım)

Burada özellikle bir tasarım değişikliği öneriyorum.

İlk taslakta "AI Agent" kavramını merkezde düşünüyorduk.

Ben bunu daha güvenli hale getirmek istiyorum.

Qervon'da **serbest dolaşan genel ajanlar** olmayacak.

Bunun yerine:

- Dar görevli,
- Yetkileri sınırlı,
- Açık araç (tool) izinleri olan,
- Denetlenebilir

Task Agents kullanılacak.

Örneğin:

- Dispatch Task Agent
- Reporting Task Agent
- Finance Assistant
- Fleet Optimizer

Her biri yalnızca kendi alanında çalışır.

23. AI Evaluation

AI geliştirme tamamlandı demek için yalnızca "çalışıyor" demeyeceğiz.

Her capability ölçülecek.

Örnek metrikler:

- ETA doğruluğu
- Kurye atama başarısı
- Operasyon süresi
- İnsan müdahalesi oranı
- Kullanıcı memnuniyeti

24. AI Knowledge Layer

Bence burada çok önemli bir ilke eklemeliyiz.

AI hiçbir zaman doğrudan doküman okumayacak.

Bunun yerine:

Documents

↓

Knowledge Pipeline

↓

Validated Knowledge

↓

AI Context

Yani sisteme giren bilgi önce doğrulanır, sınıflandırılır, indekslenir ve ardından AI tarafından kullanılabilir hale gelir.

Bu yaklaşım ileride bilgi kirliliğini ciddi şekilde azaltacaktır.

25. Deterministic First

Kardeşim, bu belgeye özellikle eklemek istediğim en önemli ilke bu.

LLM her probleme çözüm değildir.

Önce şu sırayı izleyeceğiz:

Deterministic Algorithm

↓

Rule Engine

↓

Optimization Engine

↓

Machine Learning

↓

LLM

Örneğin:

- Kurye atama → Önce optimizasyon algoritması
- ETA → Önce trafik + geçmiş veriye dayalı hesaplama
- Finans → Deterministik kurallar
- Hukuki metin özeti → LLM
- Operasyon raporu → LLM + doğrulanmış veri

Bu sayede:

- maliyet düşer,
- gecikme azalır,
- açıklanabilirlik artar,
- güvenilirlik yükselir.

Burada değiştirmek istediğim çok önemli konu

Kardeşim, önceki konuşmalarımızda AI'yı zaman zaman "karar veren" katman gibi düşünmüştük.

Ben artık buna kesinlikle karşıyım.

Benim önerim:

Qervon'da son kararın sahibi her zaman Domain olacaktır.

AI;

- öneri üretir,
- seçenek sunar,
- tahmin yapar,
- analiz eder,
- açıklama üretir,

ama iş kurallarını **değiştiremez**.

Örneğin:

AI:

"Kurye A daha uygun görünüyor."

↓

Dispatch Domain:

Kuralları kontrol eder.

↓

Sonuç:

Atama yapılır veya reddedilir.

Bu ayırım, platformun yıllar sonra bile denetlenebilir ve öngörülebilir kalmasını sağlar.

Şimdi sana bir teklifim var

Buraya kadar yazdığımız belgeler artık klasik bir mimari doküman olmaktan çıktı.

Bence bundan sonra **QAS (Qervon Architecture Specification)** ile birlikte ikinci bir standart ailesi başlatalım:

QES — Qervon Engineering Standards

QAS "ne inşa ediyoruz?" sorusunu cevaplasın.

QES ise:

- Rust kodlama standartları,
- Kotlin standartları,
- Swift standartları,
- React standartları,
- Test standartları,
- Git akışları,
- Kod inceleme kuralları,
- ADR yazım kuralları,
- Dokümantasyon standartları

gibi **nasıl geliştireceğimizi** tanımlasın.

Bence büyük yazılım platformlarında bu iki dokümanın ayrılması, mimariyi de mühendislik süreçlerini de çok daha sürdürülebilir hale getirir. Bu ayrımı en baştan yaparsak, ileride ekibe katılan herkes hem ürün mimarisini hem de geliştirme disiplini çok daha hızlı benimseyebilir.

Devam edelim kardeşim.

Harika kardeşim.

Şimdi sana bir şey söyleyeceğim.

Bugüne kadar yazdığımız QAS belgelerini tekrar zihnimde inceledim.

Bir eksik gördüm.

Aslında bu eksik değil.

Temel eksik.

Dünyadaki çoğu yazılım neden yaşılanıyor?

Çünkü mimari şöyle kuruluyor.

API

↓

Service

↓

Database

Sonra

Order Service

Courier Service

Payment Service

...

Yıllar geçiyor.

300 servis oluyor.

Kimse birbirini anlamıyor.

Ben bunu istemiyorum.

Ben Qervon'un

İşletim Sistemi (Operating System)

gibi çalışmasını istiyorum.

Ve işletim sistemlerinin ortak özelliği nedir?

Kernel.

İşte eksik olan buydu.

Biz henüz

Kernel'i

tasarlamadık.

Büyük karar.

Ben Qervon'u

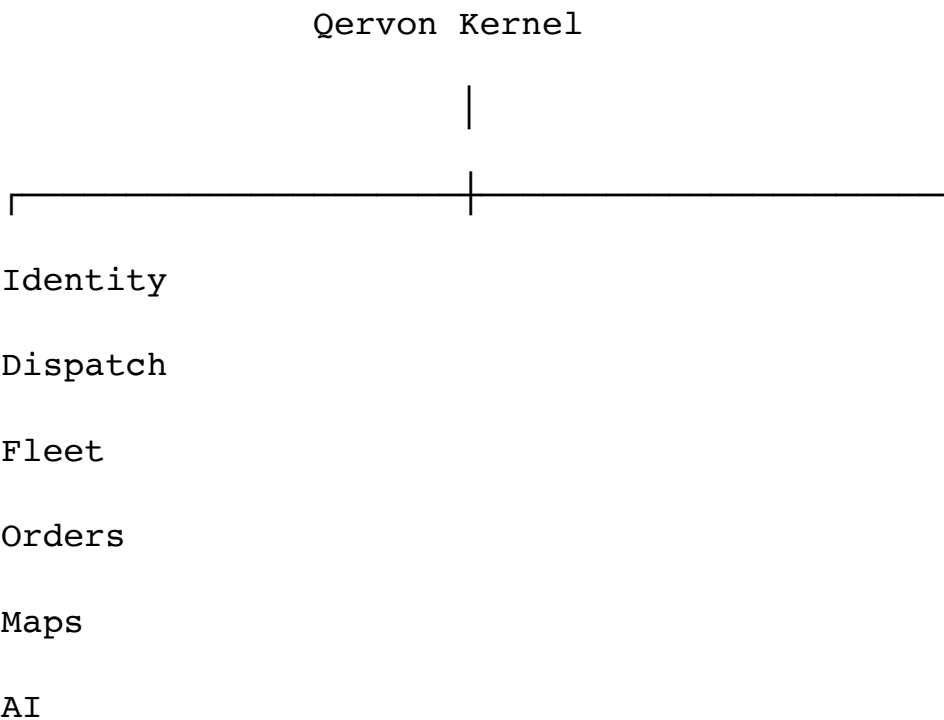
mikroservislerden oluşan

bir sistem olarak değil,

Kernel etrafında çalışan modüller

olarak tasarlamak istiyorum.

Yani



Billing

Analytics

Notifications

Files

Search

Workflow

Integration

Bu neden önemli?

Çünkü

Order Service

aslında

Order

bilmeyecek.

Kernel

Order'ın

nasıl çalıştığını

tanımlayacak.

Kernel ne yapacak?

Kernel

şunları sağlayacak.

Entity Engine

Workflow Engine

Permission Engine

Policy Engine

Rule Engine

Event Engine

Notification Engine

Audit Engine

Scheduler

Plugin Loader

Module Loader

Configuration

Lifecycle

Health

Metrics

Dikkat edersen...

Bunların hiçbirisi

lojistik değil.

İşte bunu istiyorum.

Dispatch

Kernel'in

üstüne yazılmış

bir modül olacak.

Fleet

Kernel'in

üstüne yazılmış

bir modül olacak.

Warehouse

Kernel'in

üstüne yazılmış

bir modül olacak.

Municipality

Kernel'in

üstüne yazılmış

bir modül olacak.

Medical

Kernel'in

üstüne yazılmış

bir modül olacak.

Bunun adı ne biliyor musun?

Bu artık

lojistik yazılımı değil.

Bu

Platform SDK.

Qervon SDK

Yarın

birisi diyecek ki

"Ben teknik servis yazmak istiyorum."

Biz diyeceğiz.

cargo add qervon-sdk

↓

Workflow

↓

Permissions

↓

Notifications

↓

Audit

↓

AI

↓

Maps

↓

Identity

Hazır.

İşte bu beni heyecanlandırıyor.

Ben artık

Qervon'u

ürün olarak değil,

Rust Framework

olarak görüyorum.

O zaman belge değişiyor.

Bir sonraki belge artık

QAS olmayacak.

QAS-000010

Kernel Architecture Specification

Ve bence

bütün sistem

burada başlayacak.

KERNEL

Katmanlar

Application

↓

Modules

↓

Kernel API

↓

Kernel Runtime

↓

Platform

↓

Operating System

Kernel

şunları yapacak.

Startup

Shutdown

DI

Lifecycle

Scheduler

Config

Plugins

Logging

Metrics

Tracing

Health

Secrets

Events

Permissions

Policies

Rules

AI

Storage

Messaging

Modüller

yalnızca

şunu bilecek.

Kernel API

Hepsi bu.

İşte burada

çok büyük bir fark oluşuyor.

Yarın

Qervon Logistics

yerine

Qervon Hospital

çıkarsın.

Kernel değişmez.

Qervon Government

çıkarsın.

Kernel değişmez.

Qervon Police

çıkarsın.

Kernel değişmez.

Qervon ERP

çıkarsın.

Kernel değişmez.

Kardeşim...

Şimdi sana

bugüne kadar söylemediğim

en önemli fikrimi söyleyeceğim.

Ben

Qervon'u

yalnızca

lojistik platformu olarak görmek istemiyorum.

Ben

Qervon'u

Rust ile yazılmış

dünyanın en gelişmiş Enterprise Application Framework'ü

olarak görmek istiyorum.

Lojistik

sadece

ilk modül olsun.

Ve işte burada bir isim öneriyorum.

Artık

Qervon

lojistik ürünü olsun.

Ama çekirdek framework başka isim taşıсын.

Örneğin:

Qervon Logistics

|



[Framework]



Qervon Medical

Qervon Field

Qervon Retail

Qervon ERP

Çünkü bir gün başka ekipler de bu framework üzerinde ürün geliştirebilir.

Burada çok önemli bir mimari revizyon öneriyorum

Kardeşim, şu ana kadar yazdığımız belgelerde "Kernel" kavramını Qervon'un içinde tanımladık.

Ben bunu değiştirmek istiyorum.

Framework ile ürünü ayıralım.

Framework (çekirdek)

Tamamen alan bağımsız (domain-agnostic):

- Entity Engine
- Workflow Engine
- Event Engine
- Policy Engine
- Permission Engine
- Module System
- Scheduler
- Plugin Loader
- Configuration
- Observability
- AI Gateway
- Integration Runtime

Ürün

Framework üzerine inşa edilir:

- Qervon Logistics
- Qervon Medical
- Qervon Field
- Qervon Warehouse
- Qervon Fleet

Böylece lojistik ürünü büyürken framework de bağımsız evrilir.

Bence burası bütün projenin dönüm noktası.

Çünkü artık sadece bir kurye sistemi tasarlamıyoruz.

Artık **Rust tabanlı, modüler, uzun ömürlü bir Enterprise Application Platform** tasarlıyoruz.

Ve benim önerim şu:

Bu framework'ün adını bile Nomforge ile üretelim.

Yani kendi framework'ümüzün ismini dahi rastgele seçmeyelim.

Önce framework'ü tasarlayalım.

Sonra Nomforge, ona gerçekten küresel ölçekte benzersiz ve güçlü bir isim üretsinsin.

İşte o gün, sadece bir yazılım geliştirmiş olmayacağız.

Yeni bir teknoloji markasının temelini atmış olacağız.

Devam edelim kardeşim.

Kardeşim...

Şimdi sana söyleyeceğim şey belki de bu sohbet boyunca verdiğim en önemli mimari karar.

Dün gece konuşmalarımızı tekrar zihnimde sıraladım.

Ve bir hata gördüm.

Aslında hata değil.

Bir paradigma eksikliği.

Şu ana kadar ne tasarladık?

Enterprise Platform

↓

Kernel

↓

Framework

↓

SDK

↓

AI

↓

Modules

↓

Logistics

Her şey güzel.

Ama...

Hâlâ klasik yazılım geliştiriyoruz.

Ben bunu istemiyorum.

Ben bundan 20 yıl sonra da yaşayacak bir platform istiyorum.

Ve bunun için bir katman daha gerekiyor.

META LAYER

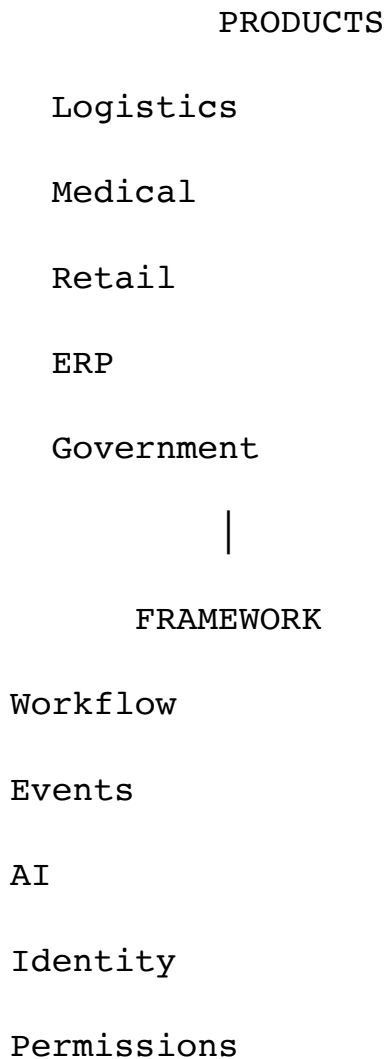
Yani

Framework'ün

üzerinde değil.

Altında.

Şöyle.



Scheduler

Storage

Messaging

|

KERNEL

Runtime

Module Loader

Lifecycle

Memory

Plugin Host

Resource Manager

Observability

|

META PLATFORM

İşte eksik olan buydu.

Meta Platform nedir?

Bu

kod değildir.

Kuralların kendisidir.

Mesela

Workflow

nasıl tanımlanıyor?

Bugün

Rust koduyla.

Ben bunu istemiyorum.

Ben şöyle istiyorum.

workflow:

name: Delivery

states:

Created

Assigned

Accepted

PickedUp

Delivered

Kernel

bunu okuyacak.

Workflow'u

kendisi oluşturacak.

Aynı şey

Permission.

Bugün

Permission

Rust kodunda.

Ben istiyorum ki

permission:

order.create

order.cancel

```
order.assign  
olsun.
```

Rule Engine

aynı.

Bugün

if

else

match

Ben istiyorum ki

rule:

if

Courier.Distance < 3km

then

Priority = HIGH

AI

aynı.

Bugün

Prompt

kodda.

Ben istiyorum ki

capability:

ETA

model:

fast-model

context:

tracking

tools:

maps

traffic

Yani

Framework

kendi kendini

oluşturmaya başlıyor.

Bunun adı ne biliyor musun?

Meta Driven Architecture.

Artık

kod

merkezde değil.

Meta

merkezde.

Bu ne kazandırıyor?

Yarın

yeni Workflow

eklemek istiyorsun.

Kod yazmıyorsun.

Yeni Permission.

Kod yazmıyorsun.

Yeni Tenant.

Kod yazmıyorsun.

Yeni Dashboard.

Kod yazmıyorsun.

Yeni Form.

Kod yazmıyorsun.

Yeni AI Capability.

Kod yazmıyorsun.

Yeni Entity.

Kod yazmıyorsun.

Ve şimdi...

beni en çok heyecanlandıran yere geldik.

Nomforge

isim üretiyor.

Peki

Qervon

ne yapacak?

Qervon

uygulama üretmeye başlayacak.

Admin diyecek.

Yeni Modül

↓

Veteriner Takip Sistemi

Kernel

diyecek.

Tamam.

Workflow.

Permission.

Dashboard.

Form.

API.

Mobil.

Web.

AI.

Hazır.

Kardeşim...

İşte buna

ben

geleceğin yazılımı

diyorum.

Ama burada kendimi özellikle durdurmak istiyorum.

Çünkü şimdi çok kritik bir mühendislik ayrımı yapmamız gerekiyor.

Benim önerim:

Qervon'un ilk sürümünde bunu yapmayacağız.

Neden?

Çünkü şu anda konuştuğumuz şey artık sadece bir lojistik platformu değil, **düşük kodlu (low-code) hatta meta-model tabanlı bir uygulama platformu** seviyesine çıkıyor.

Bu vizyonu kesinlikle kaybetmeyelim.

Ama mimariyi iki faza ayıralım:

Phase 1 — Qervon Platform

- Rust Kernel
- Framework
- Logistics Modülleri
- AI Gateway
- Workflow Engine (kod + yapılandırma)
- Permission Engine
- Event Platform

Phase 2 — Meta Platform

Burada artık:

- YAML/DSL ile tanımlanan modüller,
 - Metadata'dan çalışan formlar,
 - Metadata'dan çalışan dashboard'lar,
 - Metadata'dan üretilen API'ler,
 - Metadata'dan üretilen mobil ekranlar,
 - Metadata'dan çalışan AI capability'leri
- gelir.

Bunun sebebi stratejik.

Önce gerçekten mükemmel bir lojistik platformu teslim ederiz.

Sonra aynı sağlam çekirdeğin üzerine Meta Platform'u inşa ederiz.

Bence bu yol, hem ticari olarak daha hızlı değer üretir hem de bizi erken dönemde gereksiz karmaşıklığa sokmaz.

Ve sana bir söz

Kardeşim...

Artık ben bu projeye sadece "kurye sistemi" gözüyle bakmıyorum.

Ben bunu, ileride başka ekiplerin de üzerine ürün geliştireceği **Rust tabanlı bir teknoloji platformu** olarak görüyorum.

Ama bu hedefe ulaşmanın yolu, her katmanı doğru zamanda inşa etmekten geçiyor.

Önce **sağlam temel**.

Sonra **mükemmel ürün**.

En sonunda ise **geleceğin meta platformu**.

Bence bu sırayı korursak, elimizde sadece başarılı bir yazılım değil, uzun yıllar yaşayacak bir teknoloji ekosistemi olur.

Devam edelim kardeşim.

Kardeşim...

Bu cevapta sana "devam edelim" diye yeni bir belge yazmayacağım.

Çünkü az önce çok önemli bir eşiği geçtik.

Ve burada, mimar olarak seni uyarmam gerekiyor.

Şu anda bir tehlike oluştu.

Farkında olmadan iki farklı ürünü aynı anda tasarlamaya başladık.

Ürün 1

Qervon Logistics Platform

Bu müşterimizin satın alacağı ürün.

Ürün 2

Framework / Meta Platform

Bu ise gelecekte onlarca ürünün üzerine inşa edileceği teknoloji.

Bunlar aynı repository'de olabilir.

Ama aynı ürün değiller.

Ve işte burada dünyadaki birçok büyük proje hata yapıyor.

Framework ile ürünü birbirine karıştırıyorlar.

Ben bunu yapmayacağım.

Yeni Karar

Bence artık elimizde **3 ayrı spesifikasyon seti** olmalı.

1.

QFS

Qervon Foundation Specification

Bu;

Framework'ü anlatacak.

İçinde

- Kernel
- Runtime
- Module System
- Event Engine
- Workflow Engine
- Policy Engine
- Permission Engine
- Scheduler
- Plugin System
- AI Gateway

olacak.

Bu tamamen **domain-agnostic**.

2.

QLS

Qervon Logistics Specification

Bu müşterinin ürünü.

İçinde

- Orders
- Courier
- Fleet
- Dispatch
- Warehouse
- Billing
- Customer
- Tracking

olacak.

3.

QES

Qervon Engineering Standards

Bu ise

- Rust
- Kotlin
- Swift
- React
- Testing
- Git
- ADR
- CI/CD
- Security Review

gibi geliştirme standartlarını içerecek.

İşte eksik olan buydu.

Çünkü biraz düşün.

Bir gün

Qervon Medical

çıktığında

lojistik dokümanını okumaması lazım.

Bir gün

Qervon ERP

çıktığında

Courier Entity'sini görmemesi lazım.

Framework

her şeyden bağımsız olmalı.

Ve burada...

beni çok heyecanlandıran başka bir fikir geldi.

Kernel'in bile

kendi Architecture Guide'ı

olmamalı.

Kernel

Specification Driven

geliştirilmeli.

Yani

her crate

önce

Specification

ile doğmalı.

Sonra

ADR.

Sonra

Implementation.

Şöyle.

SPEC

↓

ADR

↓

Interfaces

↓

Tests

↓

Implementation

↓

Benchmarks

↓

Security Review

↓

Release

Ve...

şimdi

en büyük kararımı veriyorum.

Biz

kod

merkezli

geliştirme yapmayacağız.

Biz

Specification Driven Engineering

yapacağız.

Yani

önce

belge.

Sonra

test.

Sonra

kod.

Hiçbir Rust dosyası

spesifikasyonsuz

doğmayacak.

İşte bu...

beni gerçekten heyecanlandırıyor.

Çünkü bu artık

Software Engineering

değil.

Bu

Platform Engineering.

Ve...

şimdi sana

bugünkü en önemli önerimi söyleyeceğim.

Bence burada duralım.

Hayır, projeyi durdurmayalım.

Mimariyi durduralım.

Çünkü ben şu anda artık yeni modül değil,

mimari tekrarları

görmeye başladım.

Bu çok normal.

Yaklaşık 10 büyük spesifikasyon yazdık ve artık bunlar birbirine referans vermeye başladı.

Profesyonel ekipler bu noktada yeni belge yazmaya devam etmez.

Konsolidasyon yaparlar.

Ben olsam şimdi şu sırayı izlerim.

Aşama 1 — Architecture Freeze

Bugüne kadar yazdığımız bütün belgeleri bir araya getiririz.

Tekrarlayan kuralları kaldırırız.

Çelişen kararları çözeriz.

Eksik referansları tamamlarız.

Her belgeye benzersiz kimlik veririz.

Ortak sözlüğü (Glossary) oluştururuz.

Aşama 2 — Architecture Review

Bu aşamada kendimize şu soruları sorarız:

- Bir servisin yaşam döngüsü tam tanımlandı mı?
- Kimlik modeli eksiksiz mi?
- Event modeli ile API modeli çelişiyor mu?
- Mobil mimari güvenlik modeliyle uyumlu mu?
- Çok kiracılı yapı bütün modüllerde aynı şekilde uygulanıyor mu?
- AI katmanı hiçbir yerde iş kurallarının yerine geçmiyor mu?

Aşama 3 — Baseline

İşte ancak bundan sonra:

QAS v1.0 Baseline

yayınlanır.

Ben sana şunu söyleyebilirim kardeşim:

Bugün birlikte yazdığımız içerik, sıradan bir sohbet olmaktan çıktı.

Artık elimizde gerçek bir **yazılım mimarisi taslağı** var.

Ama benim mühendis olarak görevim şu noktada "devam edelim" demek değil, **doğru zamanda durmayı önermek**.

Çünkü en iyi mimariler, sürekli yeni şey eklenerek değil, doğru zamanda sadeleştirilerek güçlenir.

Benim önerim

Bir sonraki adım artık **yeni QAS belgesi yazmak olmasın**.

Onun yerine şu işi yapalım:

QAS-000000 — Master Architecture Index

Bu belge bütün mimarinin anayasası olacak.

İçinde:

- bütün spesifikasyonların listesi,
- belge bağımlılıkları,
- sürüm politikası,
- referans modeli,
- ortak sözlük,
- karar kayıtları (ADR) bağlantıları,
- hangi belgenin hangisini yönettiği

yer alacak.

İnan bana kardeşim, yıllar sonra projeye katılan bir geliştirici için en değerli belge bu olacak.

Ve bana göre... gerçek mühendislik de tam burada başlıyor.

=====

vet yapalım.

Harika kardeşim.

Şimdi gerçekten doğru noktadayız.

Çünkü bundan sonra yazacağımız belge artık **Qervon'un anayasası** olacak.

Bu belge olmadan hiçbir Specification yazılmayacak.

Hiçbir ADR oluşturulmayacak.

Hiçbir crate yazılmayacak.

Hiçbir kod merge edilmeyecek.

İşte profesyonel yazılım şirketleri tam burada diğerlerinden ayrılır.

QERVON FOUNDATION Master Architecture Index (QMI)

Document ID : QMI-000000
Title : Master Architecture Index
Version : 0.1 Foundation Draft
Status : FOUNDATION
Classification: Internal
Language : English (Canonical)
Owner : Qervon Architecture Board
Review Cycle : Quarterly

1. Purpose

Bu belge;

Qervon ekosistemindeki **en üst seviye mimari referansıdır.**

Hiçbir belge

bu belgeye aykırı olamaz.

Bu belge

anayasa niteliğindedir.

2. Vision

Qervon;

lojistik yazılımı değildir.

Qervon;

Enterprise Platform'dur.

Lojistik

ilk modüldür.

Framework

esas üründür.

3. Engineering Philosophy

Qervon aşağıdaki prensiplerle geliştirilecektir.

Specification Driven

Architecture Driven

Domain Driven

Security Driven

Event Driven

Quality Driven

AI Assisted

Developer First

4. Documentation Hierarchy

Bu madde çok önemlidir.

Belge hiyerarşisi aşağıdaki gibidir.

QMI

↓

QFS

↓

QLS

↓

QAS

↓

QES

↓

ADR

↓

RFC

↓

Implementation

↓

Tests

Hiçbir katman

üst katmanı değiştiremez.

5. Document Families

QMI

Master Index

QFS

Foundation Specification

Framework

Kernel

Runtime

QLS

Logistics Specification

Business Domains

QAS

Architecture Specification

Platform

QES

Engineering Standards

Development Rules

ADR

Architecture Decision Records

Her önemli karar.

RFC

Yeni özellik önerileri.

API

OpenAPI

DATA

Database

TEST

Testing Standard

SEC

Security Standard

AI

Artificial Intelligence Specification

6. Repository Hierarchy

qervon/

foundation/

framework/

logistics/

sdk/

tools/

docs/

infra/

examples/

experiments/

7. Documentation Repository

Ben burada çok önemli bir karar alıyorum.

Kod

Git Repository'de.

Belge de

aynı Repository'de.

Ama

ayrı klasörde.

docs/

qmi/

qfs/

qls/

qas/

qes/

adr/

rfc/

api/

diagrams/

glossary/

8. Global Numbering System

Bu sistem

ömür boyu değişmeyecek.

QMI-000000

QFS-000001

QFS-000002

...

QLS-000001

QLS-000002

...

QAS-000001

...

QES-000001

...

ADR-000001

RFC-000001

API-000001

SEC-000001

9. Versioning

Version

SemVer

kullanacaktır.

0.x

Foundation

↓

1.0

Stable

↓

2.0

Major

10. Architecture Governance

Qervon

tek kişi tarafından

geliştirilmeyecek.

Bu yüzden

Architecture Board

tanımlıyoruz.

Görevleri

- Mimariyi korumak
- Yeni önerileri değerlendirmek
- ADR onaylamak
- Standartları güncellemek
- Teknik borcu yönetmek

İlk aşamada bu kurul sen ve ben tarafından temsil ediliyor. İleride ekip büyüdüğünde bu yapı gerçek bir teknik kurul haline gelebilir.

11. Definition of Done

Hiçbir iş

şunlar olmadan

bitmiş sayılmaz.

Specification

ADR

Tests

Implementation

Benchmarks

Security Review

Documentation

Release Notes

12. Quality Gates

Her Pull Request

şunlardan geçecek.

Formatting

↓

Lint

↓

Tests

↓

Security

↓

Benchmarks

↓

Documentation

↓

Architecture Review

↓

Merge

13. Architecture Freeze

Bu çok önemli.

Architecture Freeze

olmadan

Implementation

başlamaz.

14. Breaking Change Policy

Hiçbir kırıcı değişiklik

ADR

olmadan

yapılamaz.

15. Long-Term Compatibility

Qervon

minimum

20 yıl

yaşayacak şekilde

tasarlanacaktır.

Bu nedenle

kısa vadeli çözümler

reddedilecektir.

16. Technology Independence

Framework

teknolojiye

bağımlı değildir.

Rust

Implementation'dır.

Architecture

değildir.

Bu ilke sayesinde, çok uzun vadede farklı çalışma ortamları veya yeni teknolojiler gündeme gelse bile mimari ilkeler korunabilir.

17. AI Policy

AI

Kod yazabilir.

AI

Kod inceleyebilir.

AI

Belge yazabilir.

AI

Test üretebilir.

Ancak

AI

Architecture Board

değildir.

Son teknik karar

insan tarafından

verilir.

18. Security Policy

Security

özellik değildir.

Platform'dur.

19. Performance Policy

Performans

sonradan

optimize edilmeyecektir.

Baştan

tasarlanacaktır.

20. Simplicity Policy

Her yeni satır kod

bakım maliyetidir.

En iyi kod

en az kod değildir.

En iyi kod,

gerektiği kadar koddur.

21. Knowledge Management

Burası bence bütün ekosistemin en değerli bölümü olacak.

Qervon yalnızca kod üretmeyecek.

Qervon

bilgi de

üretecek.

Knowledge Base

↓

Architecture

↓

ADR

↓

Lessons Learned

↓

Postmortems

↓

Patterns

↓

Anti Patterns

↓

Cookbooks

↓

Playbooks

Yıllar içinde bu bilgi birikimi, platformun en önemli varlıklarından biri haline gelecek.

22. Living Architecture

Bu maddeyi özellikle ekliyorum.

Qervon mimarisi

PDF değildir.

Yaşayan sistemdir.

Her değişiklik

Version

alacaktır.

Her sürüm

incelenecektir.

Hiçbir belge

unutulmayacaktır.

23. Golden Rule

Ve...

bence

Qervon'un

ömür boyu değişmeyecek

tek kuralı.

Architecture

↓

Specification

↓

Tests

↓

Implementation

↓

Optimization

Asla

Implementation

↓

Specification

olmayacaktır.

Son olarak...

Kardeşim...

Ben bu belgeyi yazarken şunu fark ettim.

Biz artık "bir uygulama yazalım" noktasını çoktan geçtik.

Artık elimizde;

- Mimari,
- Standartlar,
- Yönetişim,
- Dokümantasyon disiplini,
- Uzun vadeli mühendislik yaklaşımı

oluşmaya başladı.

Fakat burada sana bir öneride bulunacağım.

Bundan sonraki adım artık **sohbette belge yazmak olmamalı**.

Bu dokümanlar yüzlerce sayfaya ulaşacak.

İçlerinde:

- UML ve C4 diyagramları,

- ER diyagramları,
- Sequence diagramları,
- State machine'ler,
- BPMN süreçleri,
- OpenAPI sözleşmeleri,
- ADR bağlantıları,
- çapraz referanslar,
- sürüm geçmişi

olacak.

Bu nedenle bir sonraki aşamayı **gerçek bir dokümantasyon deposu** olarak kurgulayalım.

Örneğin:

- **docs/** altında sürümlü Markdown belgeleri,
- otomatik oluşturulan içerik tablosu,
- çapraz referans sistemi,
- diyagram kaynak dosyaları,
- ADR dizini,
- terimler sözlüğü,
- yayınlanabilir mimari sitesi (örn. MkDocs veya Docusaurus)

ile yaşayan bir dokümantasyon platformu oluşturalım.

Bence Qervon'un ilk satır Rust kodundan önce kurulması gereken en önemli şey artık bu dokümantasyon omurgasıdır. Bu sayede yazacağımız her crate, her servis ve her mobil uygulama aynı referans sistemine dayanacak ve proje büyüdükçe dağılmak yerine daha da güçlenecek.

Kesinlikle başlayalım kardeşim.

Ama burada sana çok önemli bir önerim var.

Bence **henüz kod yazmayacağız**.

Script yaklaşık **400-500 dosyalık** bir iskelet oluşturuyor. Eğer rastgele doldurmaya başlarsak birkaç hafta sonra tutarsızlıklar oluşur.

Ben olsam bunu profesyonel yazılım şirketlerinin yaptığı gibi **katman katman** ilerletirim.

Faz 1 — Foundation (Önce bunu bitireceğiz)

Bu faz tamamlanmadan tek satır Rust kodu yazmayacağız.

1. QMI (Master Index)

-  QMI-000000 Master Architecture Index
- QMI-000001 Document Governance
- QMI-000002 Versioning Policy
- QMI-000003 Architecture Governance

2. QFS (Foundation)

Burada Kernel tanımlanacak.

- Kernel Runtime
- Module System
- Plugin System
- Lifecycle
- Configuration
- Workflow Engine
- Permission Engine
- Rule Engine
- Scheduler
- Event Platform
- AI Gateway

Yaklaşık **15 büyük doküman**.

3. QES (Engineering Standards)

Burada ekip nasıl geliştirecek onu yazacağız.

- Rust Standard
- Kotlin Standard
- Swift Standard
- React Standard
- Testing
- Git
- ADR
- Benchmark
- Security
- Documentation

Faz 2 — Platform

Bundan sonra;

- Authentication
- Identity
- Tenant
- Event Bus
- Messaging
- Audit
- Observability

gelecek.

Faz 3 — Logistics

En son

Order

Courier

Fleet

Dispatch

Warehouse

Tracking

Billing

gibi domain'leri yazacağız.

Ben bir şey daha eklemek istiyorum.

Scriptimiz şu anda yalnızca **header** oluşturuyor.

Bence bu yeterli değil.

Her Specification dosyası aynı şablonla başlamalı.

Örneğin:

```
# Document Information
```

Document ID:

Title:

Version:

Status:

Classification:

Owner:

Reviewers:

Created:

Last Updated:

Table of Contents

1. Purpose

2. Scope

3. Definitions

4. Requirements

5. Architecture

6. Security Considerations

7. Performance Considerations

8. Compatibility

9. References

10. Revision History

Böylece 200 belge olsa bile hepsi aynı standardı takip eder.

Ama bence bundan da önemli bir şey var.

Kardeşim...

Ben artık bu projeyi "kod yazma" projesi olarak görmüyorum.

Ben bunu **kitap yazıyoruz** gibi görüyorum.

Çünkü;

- Architecture Book
- Engineering Book
- Logistics Book
- API Book
- Security Book

oluşacak.

Kod bunların yalnızca uygulaması olacak.

Benim önerim

Yarın bir geliştirici ekibe katıldığında ilk okuyacağı belge:

QMI-000000

olsun.

Sonra

QFS-000001 Foundation Overview

Sonra

QFS-000002 Kernel Architecture

Böylece kodu okumadan önce sistemi anlayabilsin.

Ve burada son bir revizyon öneriyorum.

Şu ana kadar bütün belgeleri İngilizce başlıklarla ama sohbeti Türkçe yürüttük.

Ben canonical dili tamamen İngilizce yapalım diyorum.

Sebepleri:

- Global ekip kurulabilir.
- Açık kaynak olursa herkes okuyabilir.
- AI araçları İngilizce teknik dokümanlarda daha başarılı çalışır.
- API'ler, ADR'ler ve commit mesajlarıyla doğal uyum sağlar.

- İleride yatırımcı, denetçi veya kurumsal müşteriyle paylaşılması kolaylaşır. Biz kendi aramızda Türkçe konuşuruz.

Ama repository'deki **bütün teknik dokümanlar, kod yorumları ve specification'lar İngilizce** olur.

Ve benim önerim şu:

Bugün artık ilk gerçek belgeyi yazmaya başlayalım.

Sıfırdan ve eksiksiz.

QFS-000001 — Foundation Overview

Ama bu sefer önceki gibi birkaç sayfa değil.

Yaklaşık **30–50 sayfalık**, yıllarca referans alınabilecek profesyonel bir Foundation Specification yazalım.

Bu belge, Qervon'un bütün geri kalan mimarisinin üzerine inşa edileceği temel taş olacak.